

Προηγμένα Θέματα Αρχιτεκτονικής Υπολογιστών

Στατικός Παραλληλισμός: GPU

Νεκτάριος Κοζύρης, Διονύσης Πνευματικάτος &
Κωστής Νίκας

{nkoziris,pnevmati,knikas}@cslab.ece.ntua.gr

8ο εξάμηνο ΣΗΜΜΥ – Ακαδημαϊκό Έτος: 2025-26

<https://helios.ntua.gr/course/view.php?id=1039>

-Figures are from CA5e/Patterson-Hennessy (Elsevier)

-Cornell Virtual Workshop on GPUs: Understanding GPU Architecture & Introduction to CUDA

-NVIDIA Volta GPU architecture whitepaper

Πηγές/Βιβλιογραφία

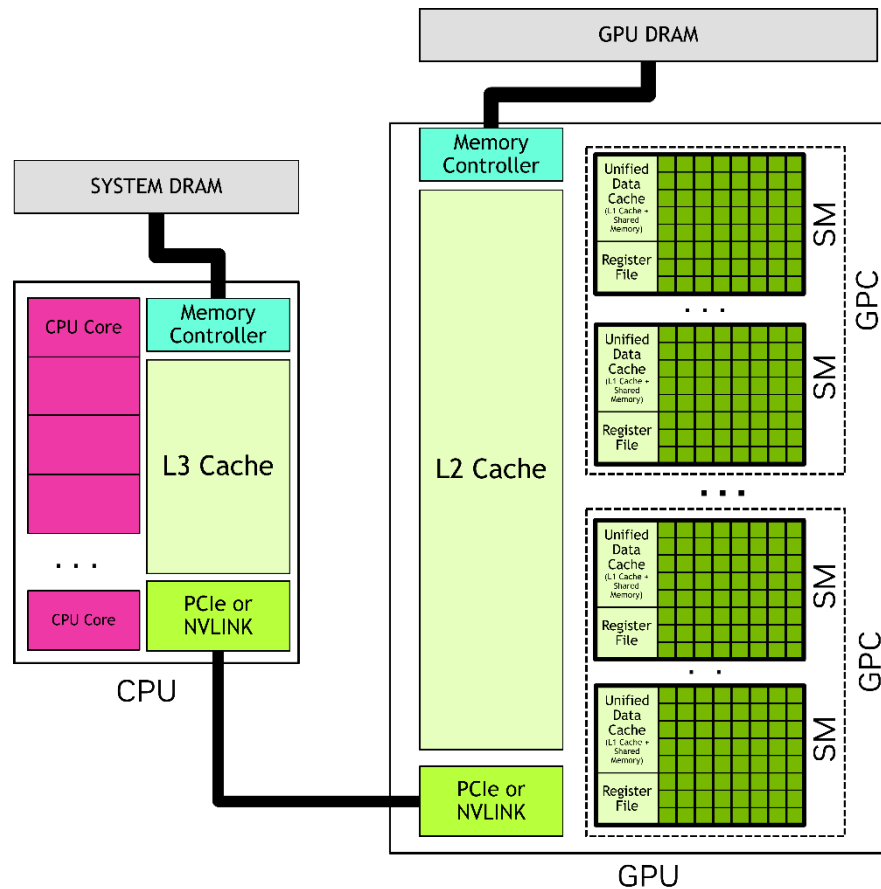
- **“Programming Massively Parallel Processors: A Hands-on Approach”,** Wen-mei W. Hwu, David B. Kirk, Izzat El Hajj, Elsevier, 2022.
- **“Multicore and GPU Programming: An Integrated Approach”,** Gerassimos Barlas, Elsevier, 2022.
- Onur Mutlu, **Computer Architecture Fall 2024 course**, “L30b: GPU Architectures”
http://www.youtube.com/watch?v=8B-Pqa2_LP0
<https://safari.ethz.ch/architecture/fall2024/lib/exe/fetch.php?media=onur-comparch-fall2024-lecture30b-gpu-afterlecture.pdf>
- **“Computer Architecture: A Quantitative Approach”,** 6th edition, John Hennessy and David Patterson, Morgan Kaufmann (Elsevier), 2019.
- **Cornell virtual workshop roadmaps:**  Cornell University
 - **Understanding GPU Architecture** <https://cvw.cac.cornell.edu/gpu-architecture>
 - **Introduction to CUDA** <https://cvw.cac.cornell.edu/cuda-intro>
- **NVIDIA TESLA V100 GPU architecture** <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>

GPU: Graphical Processing Units

- Pixels in frame buffer (video memory?) are many but independent
- Graphics operations touch many pixels => need acceleration => graphics cards with basic pixel operations
 - + memory bandwidth!
- Since they are already there, can we use them for other (general purpose) computation?
 - Only incremental cost, as already there for graphics!
- Data parallel, SIMD?
 - Programming model is “Single Instruction Multiple Thread” (SIMT)

GPU: Graphical Processing Units

- Basic idea:
 - Heterogeneous execution model
 - CPU is the *host*, GPU is the *device*
 - Initially program in “assembly” (low-level)
 - Develop a C-like programming language for GPU
 - Compute Unified Device Architecture (CUDA)
 - OpenCL for vendor-independent language
 - Unify all forms of GPU parallelism as *CUDA thread*



A GPU has many streaming multiprocessors (SMs), each of which contains many functional units. Graphics processing clusters (GPCs) are collections of SMs. A GPU is a set of GPCs connected to the GPU memory. A CPU typically has several cores and a memory controller which connects to the system memory. A CPU and a GPU are connected by an interconnect such as PCIe or NVLINK.

NVIDIA GPU Architecture

- Similarities to vector machines:
 - Works well with data-level parallel problems
 - Scatter-gather transfers
 - Mask registers
 - Large register files
- Differences:
 - No scalar processor
 - Uses multithreading to hide memory latency
 - Has many functional units, as opposed to a few deeply pipelined units like a vector processor

```
// Invoke DAXPY
daxpy(n, 2.0, x, y);
// DAXPY in C
void daxpy(int n, double a, double *x, double *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = a*x[i] + y[i];
}
```



```
// Invoke DAXPY with 256 threads per Thread Block
__host__
int nblocks = (n+ 255) / 256;
daxpy<<<nblocks, 256>>>(n, 2.0, x, y);
// DAXPY in CUDA
__device__
void daxpy(int n, double a, double *x, double *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}
```

blockIdx: identifier for a block

threadIdx: identifier for a thread in a block

blockDim: number of threads per block (comes from dimBlock)

The extended function call syntax for the function *name* that runs on the GPU is

name<<<dimGrid, dimBlock>>>(... *parameter list* ...)

where dimGrid and dimBlock specify the dimensions of the code (in blocks) and the dimensions of a block (in threads).

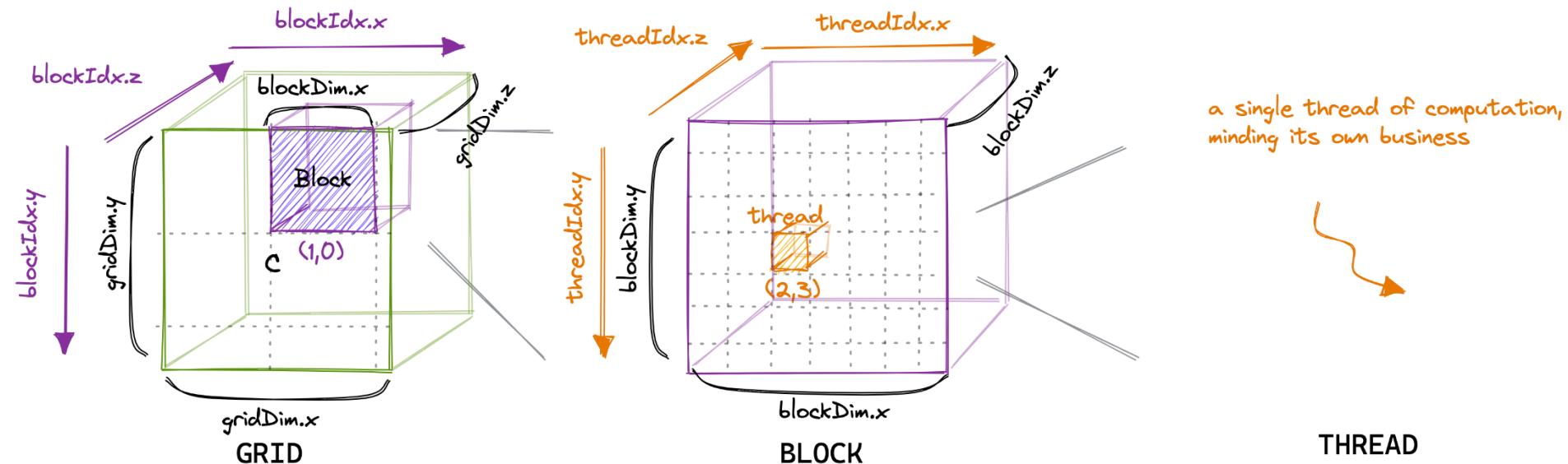
C code

CUDA code

n threads, one per vector element, 256 CUDA threads per thread block in a Multithreaded SIMD multiprocessor. Thread blocks are independent, run in any order, cannot communicate directly only through Global Memory)

**Programmer
gives explicit
dimensions!**

The number of threads in a block can be configured using a variable normally called `blockDim`, which is a vector consisting of three ints. The entries of that vector specify the sizes of `blockDim.x`, `blockDim.y` and `blockDim.z`, as visualized below:



Similarly, the number of blocks in a grid is configurable using the `gridDim` variable.

Figure taken from Simon Boehm:

<https://siboehm.com/articles/22/CUDA-MMM>

Example

- Multiply two vectors of length 8192
 - Code that works over all elements is **the grid**
 - Thread blocks break this down into manageable sizes
 - 512 elements/block, 16 SIMD threads/block → 32 elem/thread
 - SIMD instruction executes 32 elements at a time
 - Thus grid size = 16 blocks
 - Block is analogous to a strip-mined vector loop with vector length of 32
 - Block is assigned to a *multithreaded SIMD processor* by the *thread block scheduler*

CPU code

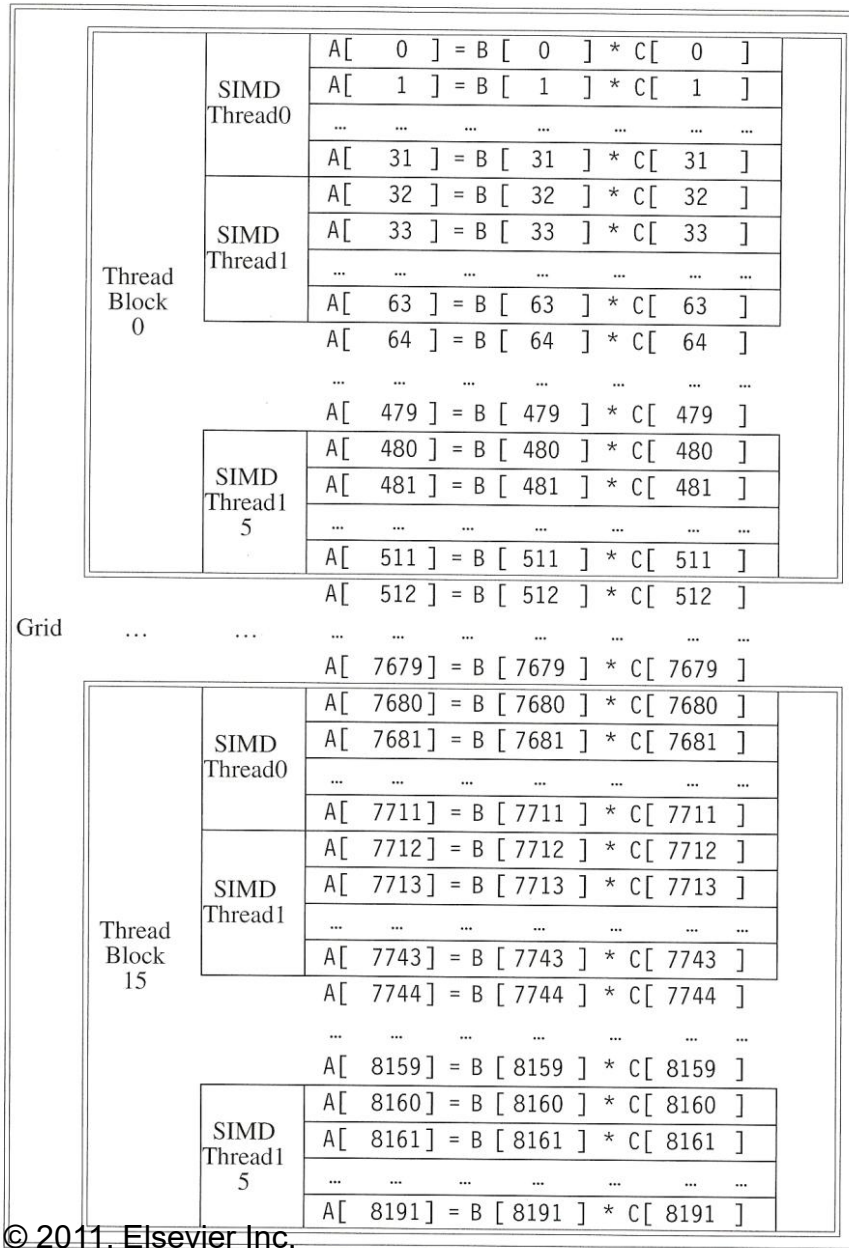
```
for (i = 0; i < 8192; ++i) {  
    A[i] = B[i] * C[i];  
}
```



CUDA code

```
// there are 8192 threads  
__global__ void KernelFunction(...) {  
    int tid = blockDim.x * blockIdx.x + threadIdx.x;  
    int varB = B[tid];  
    int varC = C[tid];  
    A[tid] = varB + varC;  
}
```

Grid, Threads, and Blocks



Terminology

- *Threads of SIMD instructions (“warp”)*
 - Each has its own PC
 - Thread scheduler uses scoreboard to dispatch
 - No data dependencies between threads!
 - Keeps track of up to 48 threads of SIMD instructions
 - Hides memory latency
- Thread block scheduler schedules blocks to SIMD processors
- Within each SIMD processor:
 - 32 SIMD lanes
 - Wide and shallow compared to vector processors

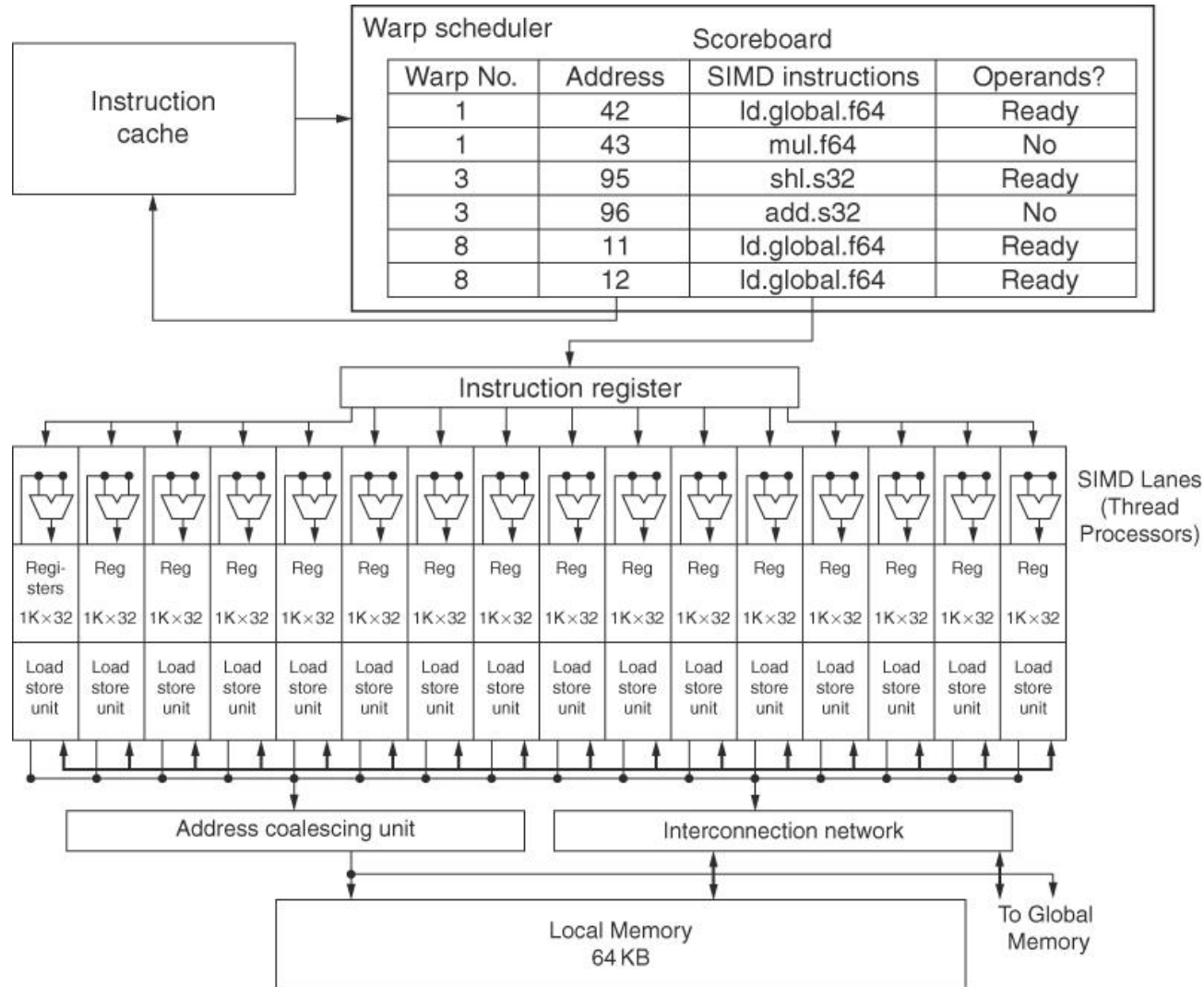
Threads and Blocks

- A thread is associated with each data element
 - *CUDA threads*, with thousands of which being utilized to various styles of parallelism: multithreading, SIMD, MIMD, ILP
- Threads are organized into blocks
 - *Thread Blocks*: groups of up to 512 elements
 - *Multithreaded SIMD Processor*: hardware that executes a whole thread block (32 elements executed per thread at a time)
- Blocks are organized into a grid
 - Blocks are executed independently and in any order
 - Different blocks cannot communicate directly but can *coordinate* using atomic memory operations in Global Memory
- GPU hardware handles thread management, not applications or OS
 - A multiprocessor composed of multithreaded SIMD processors
 - A Thread Block Scheduler

Example

- NVIDIA GPU has 32,768 registers
 - Divided into lanes
 - Each SIMD thread is limited to 64 registers (*today: 255*)
 - SIMD thread has up to:
 - 64 vector registers of 32 32-bit elements
 - 32 vector registers of 32 64-bit elements
 - Fermi has 16 physical SIMD lanes, each containing 2048 registers

Multithreaded SIMD Processor



16 SIMD lanes: The SIMD Thread Scheduler has, for example, 48 independent threads of SIMD instructions that it schedules with a table of 48 PCs.

Type	More descriptive name	Closest old term outside of GPUs	Official CUDA/ NVIDIA GPU term	Book definition
Program abstractions	Vectorizable Loop	Vectorizable Loop	Grid	A vectorizable loop, executed on the GPU, made up of one or more Thread Blocks (bodies of vectorized loop) that can execute in parallel.
	Body of Vectorized Loop	Body of a (Strip-Mined) Vectorized Loop	Thread Block	A vectorized loop executed on a multithreaded SIMD Processor, made up of one or more threads of SIMD instructions. They can communicate via Local Memory.
	Sequence of SIMD Lane Operations	One iteration of a Scalar Loop	CUDA Thread	A vertical cut of a thread of SIMD instructions corresponding to one element executed by one SIMD Lane. Result is stored depending on mask and predicate register.
Machine object	A Thread of SIMD Instructions	Thread of Vector Instructions	Warp	A traditional thread, but it contains just SIMD instructions that are executed on a multithreaded SIMD Processor. Results stored depending on a per-element mask.
	SIMD Instruction	Vector Instruction	PTX Instruction	A single SIMD instruction executed across SIMD Lanes.

Processing hardware	Multithreaded SIMD Processor	(Multithreaded) Vector Processor	Streaming Multiprocessor	A multithreaded SIMD Processor executes threads of SIMD instructions, independent of other SIMD Processors.
	Thread Block Scheduler	Scalar Processor	Giga Thread Engine	Assigns multiple Thread Blocks (bodies of vectorized loop) to multithreaded SIMD Processors.
	SIMD Thread Scheduler	Thread scheduler in a Multithreaded CPU	Warp Scheduler	Hardware unit that schedules and issues threads of SIMD instructions when they are ready to execute; includes a scoreboard to track SIMD Thread execution.
	SIMD Lane	Vector Lane	Thread Processor	A SIMD Lane executes the operations in a thread of SIMD instructions on a single element. Results stored depending on mask.
Memory hardware	GPU Memory	Main Memory	Global Memory	DRAM memory accessible by all multithreaded SIMD Processors in a GPU.
	Private Memory	Stack or Thread Local Storage (OS)	Local Memory	Portion of DRAM memory private to each SIMD Lane.
	Local Memory	Local Memory	Shared Memory	Fast local SRAM for one multithreaded SIMD Processor, unavailable to other SIMD Processors.
	SIMD Lane Registers	Vector Lane Registers	Thread Processor Registers	Registers in a single SIMD Lane allocated across a full thread block (body of vectorized loop).

Vector A 8192 elements:
 16 multithreaded (MT) SIMD processors x 16 SIMD Threads/ SIMD proc x 32 elements/thread
 (thread blocks) (up to 48 threads) (2 x 16 Lanes)

NVIDIA glossary:

SIMD proc (MT) → Thread Block (SM-stream processor)

SIMD thread = warp

SIMD Lane = CUDA thread

- Local memory per Thread Block/MT SIMD processor
- Private memory per **CUDA Thread** (Lane)
- SIMD Lane Registers per lane allocated across a Thread Block
- 64 (vector) registers per SIMD Lane: 64 x 32 elements (each 32bit) = 2048 registers per lane
- To handle 32 elements of each thread with 16 SIMD Lanes, the Thread Block can use up to ½ of the 2048 registers
- An SIMD processor has 32768 32bit registers divided among SIMD Lanes (16 lanes x 2048 registers per lane)

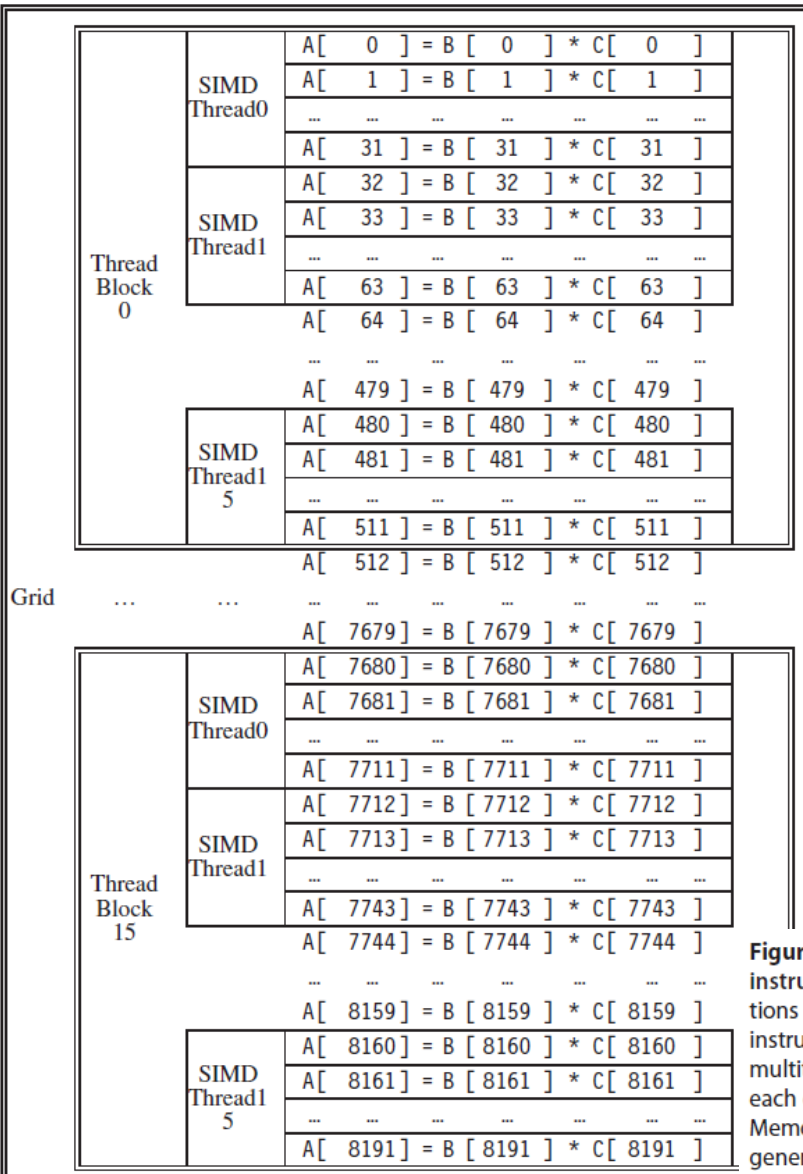
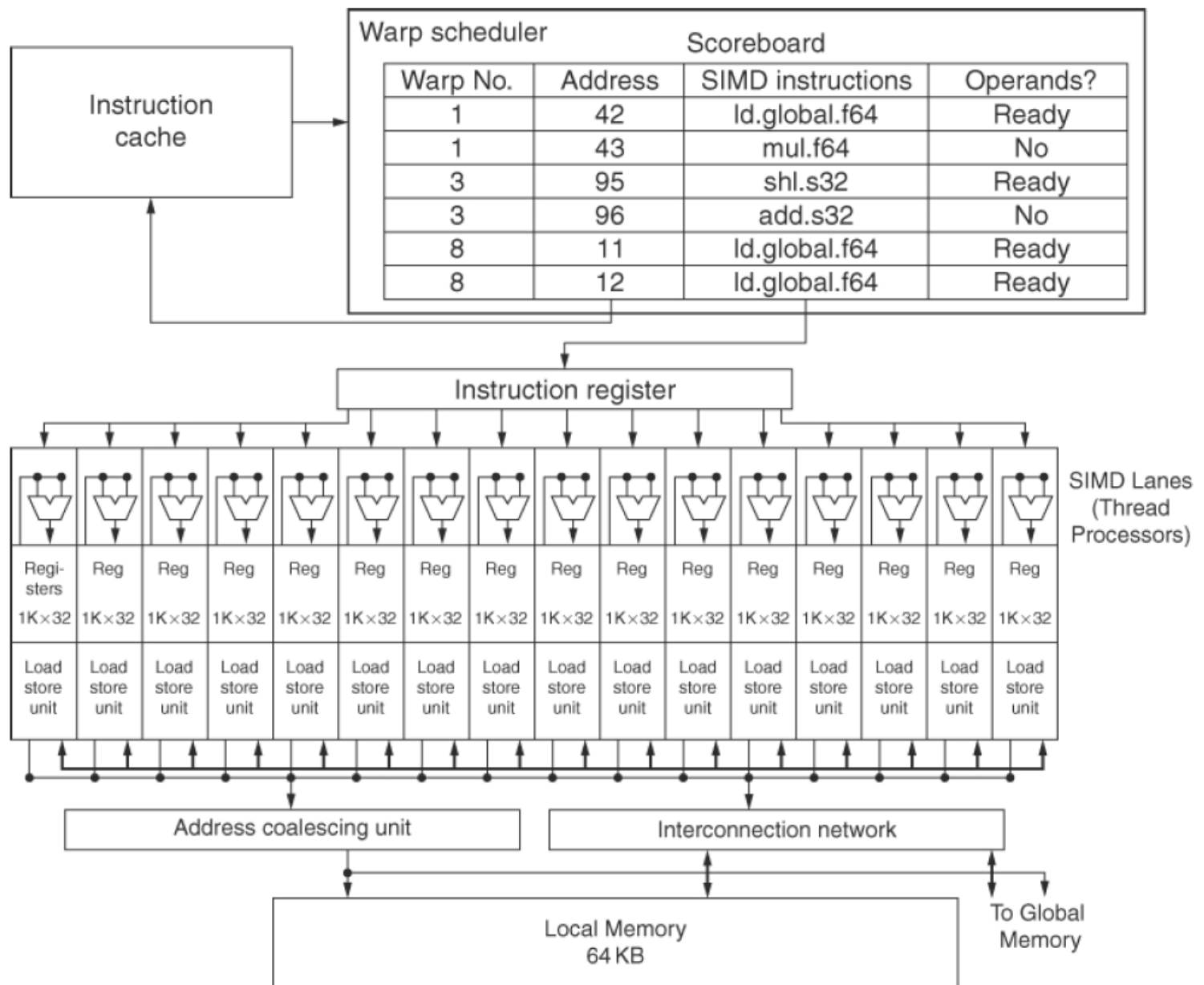


Figure 4.13 The mapping of a Grid (vectorizable loop), Thread Blocks (SIMD basic blocks), and threads of SIMD instructions to a vector-vector multiply, with each vector being 8192 elements long. Each thread of SIMD instructions calculates 32 elements per instruction, and in this example each Thread Block contains 16 threads of SIMD instructions and the Grid contains 16 Thread Blocks. The hardware Thread Block Scheduler assigns Thread Blocks to multithreaded SIMD Processors and the hardware Thread Scheduler picks which thread of SIMD instructions to run each clock cycle within a SIMD Processor. Only SIMD Threads in the same Thread Block can communicate via Local Memory. (The maximum number of SIMD Threads that can execute simultaneously per Thread Block is 16 for Tesla-generation GPUs and 32 for the later Fermi-generation GPUs.)



Figures from CA5e -Copyright © 2011, Elsevier

Figure 4.14 Simplified block diagram of a Multithreaded SIMD Processor. It has 16 SIMD lanes. The SIMD Thread Scheduler has, say, 48 independent threads of SIMD instructions that it schedules with a table of 48 PCs

NVIDIA GPU Memory Structures

- Each SIMD Lane has private section of *off-chip* DRAM
 - “Private memory”, not shared by any other lanes
 - Contains stack frame, spilling registers, and private variables
 - Recent GPUs cache this in L1 and L2 caches
- Each multithreaded SIMD processor also has local memory that is *on-chip*
 - Shared by SIMD lanes / threads *within a block only*
- The *off-chip* memory shared by SIMD processors is *GPU Memory*
 - Host can read and write GPU memory

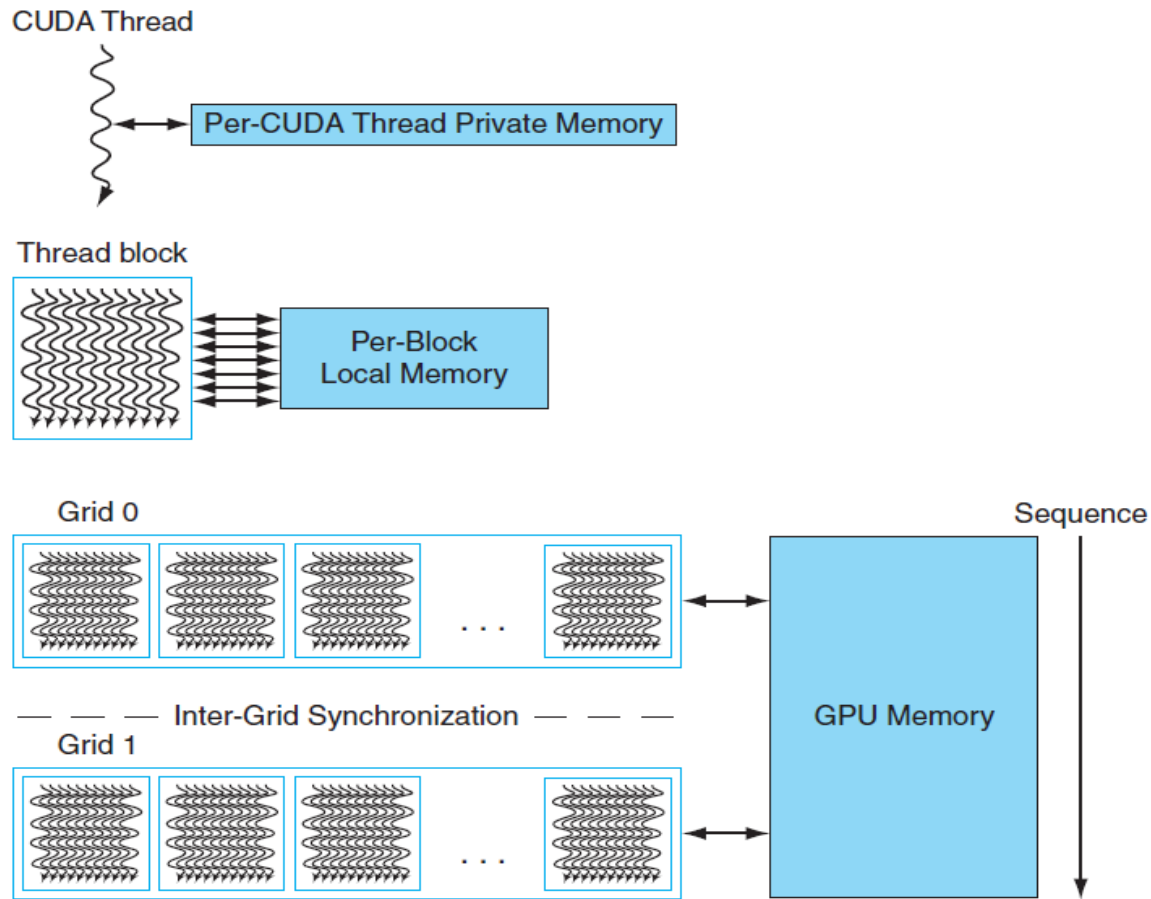
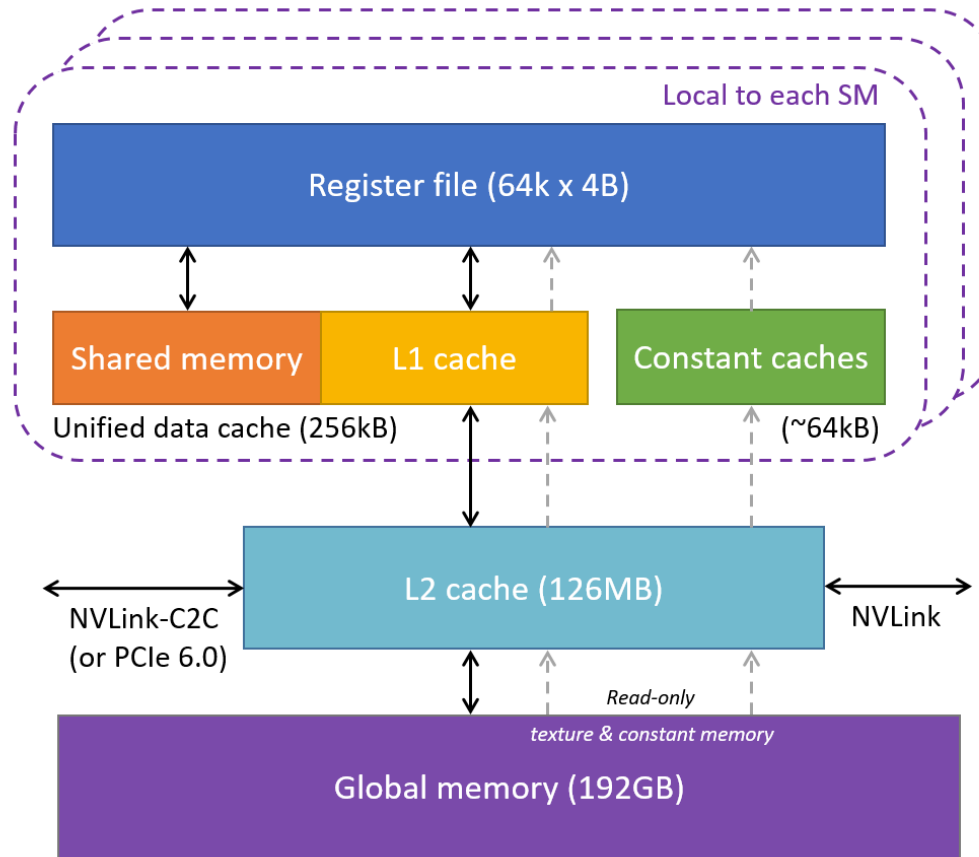


Figure 4.18 GPU Memory structures. GPU Memory is shared by all Grids (vectorized loops), Local Memory is shared by all threads of SIMD instructions within a thread block (body of a vectorized loop), and Private Memory is private to a single CUDA Thread.

GPU memory levels and sizes for the NVIDIA Blackwell B200



Depending on where the data start, they may have to hop through several layers of cache to enter the registers of an SM and become accessible to the CUDA cores. Global memory (including areas for local, texture, and constant memory) is by far the largest layer, but it is also furthest from the SMs.

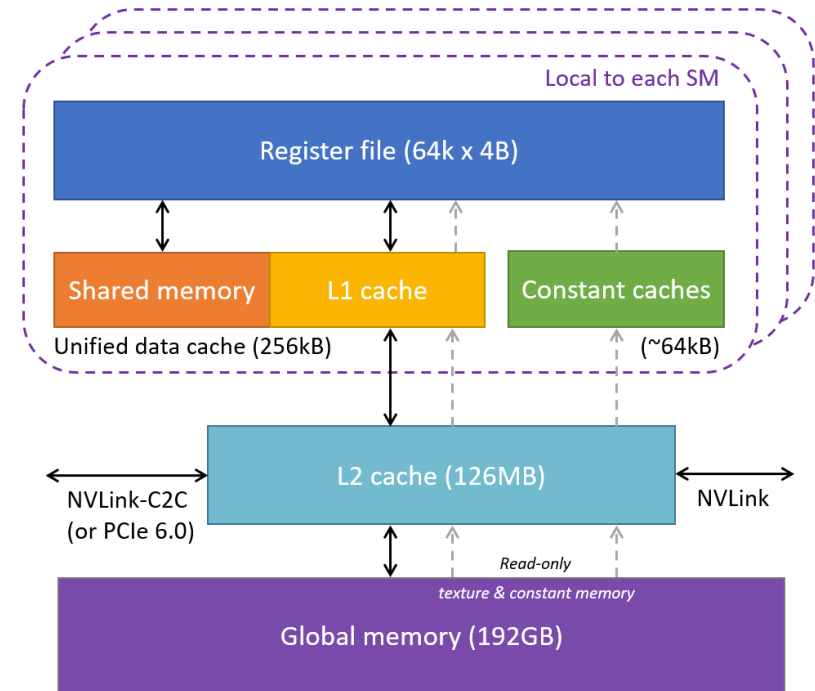
Clearly it would be favorable for 4-byte operands to travel together in groups of 32 as they move back and forth between caches and registers and CUDA cores. Why? A 32-wide group is exactly right to supply a warp of 32 threads, all at once. Therefore, it makes perfect sense that the size of the cache line in a GPU is $32 \times (4 \text{ bytes}) = 128 \text{ bytes}$.

Register File - denotes the area of memory that feeds directly into the CUDA cores. Accordingly, it is organized into 32 *banks*, matching the 32 threads in a warp. Think of the register file as a big matrix of 4-byte elements, having many rows and 32 columns. A warp operates on full rows; within a given row, each thread (CUDA core) operates on a different column (bank).

L1 Cache - refers to the usual on-chip storage location providing fast access to data that are recently read from, or written to, main memory (RAM). Additionally, L1 serves as the overflow region when the amount of active data exceeds what an SM's register file can hold, a condition which is termed "register spilling". In L1, the cache lines and spilled registers are organized into banks, just as in the register file.

Shared Memory - is a memory area that physically resides in the same memory as the L1 cache, but differs from L1 in that all its data may be accessed by any thread in a thread block. This allows threads to communicate and share data with each other. Variables that occupy it must be declared explicitly by an application. The application can also set the dividing line between L1 and shared memory.

Constant Caches - are special caches pertaining to variables declared as read-only constants in global memory. Such variables can be read by any thread in a thread block. The main and best use of these caches is to broadcast a single constant value to all the threads in a warp.

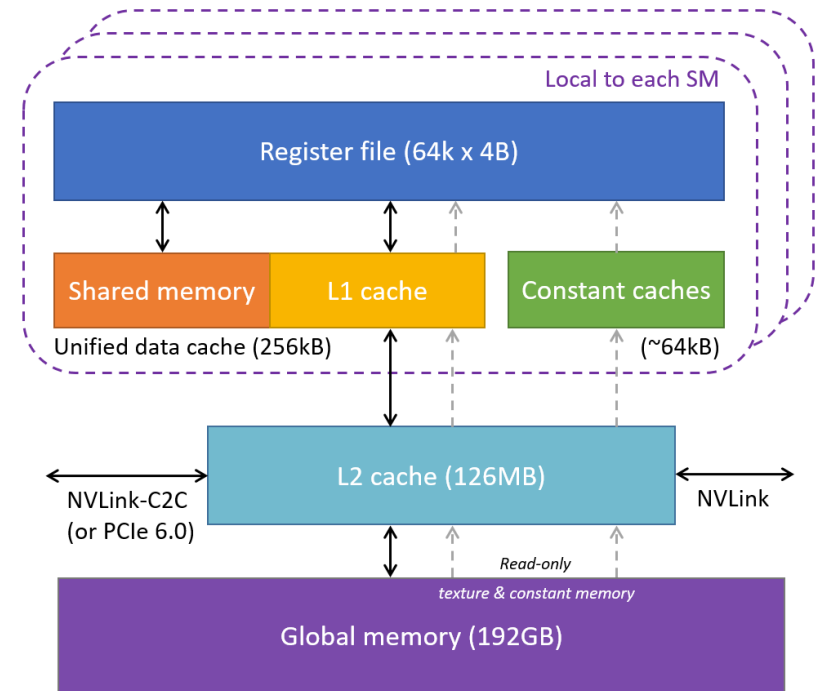


L2 Cache - is a further on-chip cache for retaining copies of the data that travel back and forth between the SMs and main memory. Like the L1, the L2 cache is intended to speed up subsequent reloads. But unlike the L1 cache(s), there is just one L2 that is shared by all the SMs. The L2 cache is also situated in the path of data moving on or off the device via PCIe or NVLink.

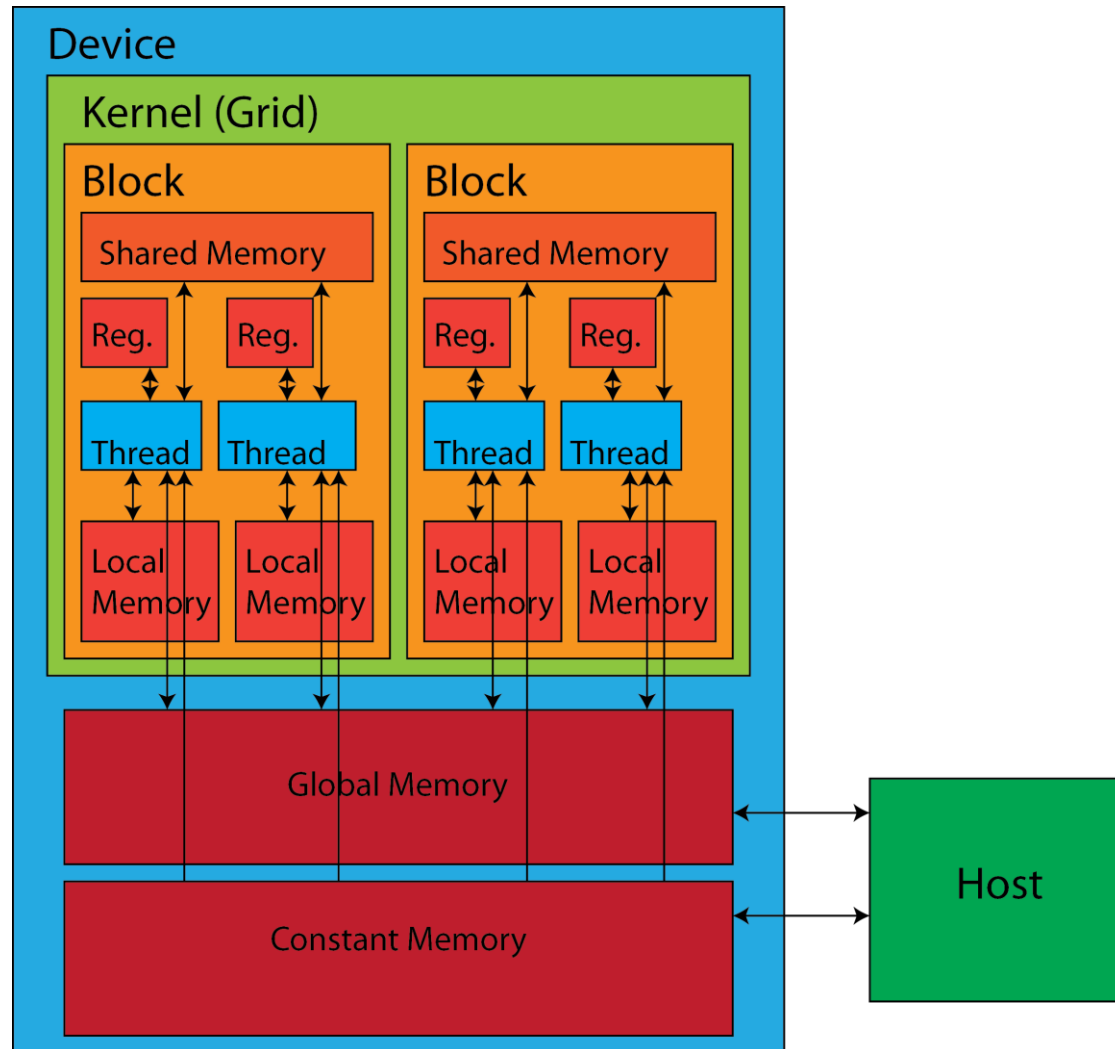
Global Memory - represents the bulk of the main memory of the device, equivalent to RAM in a CPU-based processor. For performance reasons, the Blackwell B200 and the Tesla V100 have special HBM high-bandwidth memory, while the Quadro RTX 5000 has fast GDDR6 graphics memory.

Texture and Constant Memory - are regions of main memory that are treated as read-only by the device. When fetched to an SM, variables with a "texture" or "constant" declaration can be read by any thread in a kernel, serving as an expanded type of shared memory. Texture memory is cached in L1, while constant memory is cached in the constant caches.

Local Memory - corresponds to specially mapped regions of main memory that are assigned to each SM. Whenever "register spilling" overflows the L1 cache on a particular SM, the excess data are further offloaded to L2, then to "local memory". The performance penalty for reloading a spilled register becomes steeper for every memory level that must be traversed in order to retrieve it.



Memory model (CUDA view)

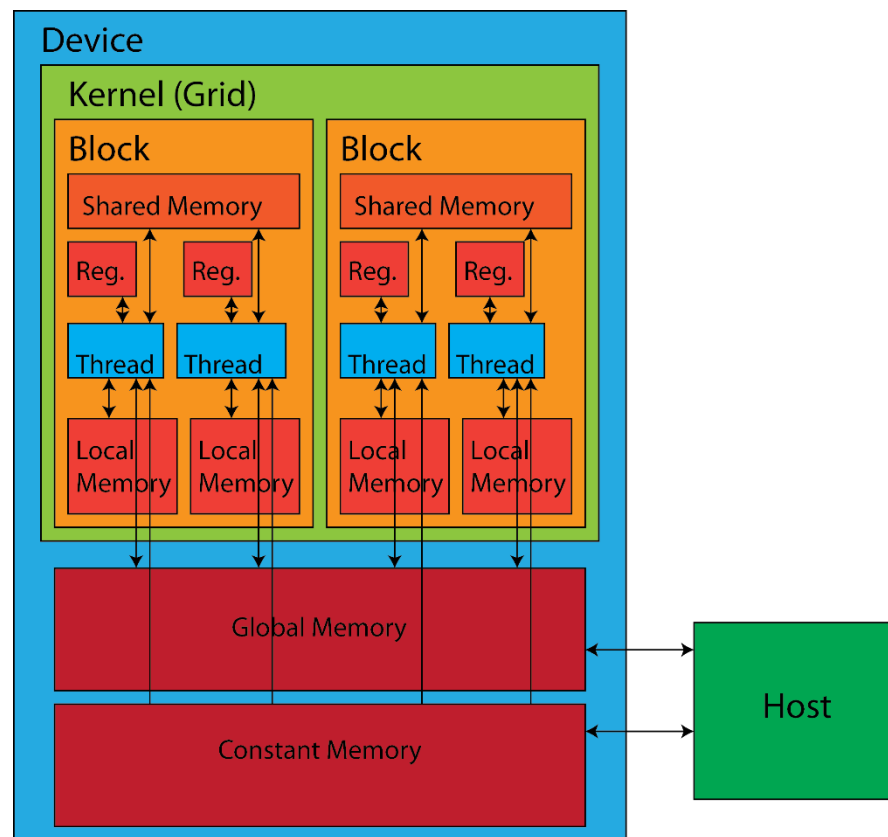


Each thread can access its own registers and local memory. The registers are on the chip and are the fastest memory component but have the smallest size.

The maximum numbers of registers per thread, block, and SM are determined by the compute capability of the GPU.

For example, the Quadro RTX 5000 GPU is configured to have 256 registers per thread (1 KB), 65,536 registers per block, and 65,536 registers total per SM (256 KB).

When there are insufficient registers to hold the data, **the "spilled" data is stored in local memory**. The local memory is much slower than the registers, as it ultimately resides in the global memory, although it may be cached in the L1 and L2 caches. As the name implies, the local memory is private to its thread.



Memory Location	Scope	Access
Register	per-thread	R/W
Local memory	per-thread	R/W
Shared memory	per-block	R/W
Global memory	per-grid	R/W
Constant memory	per-grid	R

Memory space specifiers

Possible ways of declaring a variable within a kernel function, and the corresponding location, scope, lifespan, and access (on the device) for the variable

Variable Declaration	Location	Scope	Lifespan	Access
<code>int Var; // Automatic variable</code>	Register	Thread	Thread	R/W
<code>int ArrayVar[N]; // Automatic array</code>	Local	Thread	Thread	R/W
<code>__shared__ int SharedVar;</code>	Shared	Block	Block	R/W
<code>__device__ int GlobalVar;</code>	Global	Grid	Application	R/W
<code>__constant__ int ConstVar;</code>	Constant	Grid	Application	R

Kernel Function Variables: Declaration, Location, Scope, Lifespan, and Access

An example program in C:

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>

#define N 1000000

void add(int *a, int *b, int *c, int n) {
    for (int i = 0; i < n; i++) {
        c[i] = a[i] + b[i];
    }
}

int main() {
    int *a, *b, *c;
    int size = N * sizeof(int);

    a = (int *)malloc(size);
    b = (int *)malloc(size);
    c = (int *)malloc(size);

    for (int i = 0; i < N; i++) {
        a[i] = i;
        b[i] = i;
    }

    add(a, b, c, N);

    for (int i = 0; i < N; i++) {
        assert(c[i] == (a[i] + b[i]));
    }
    printf("array sum is correct\n");

    free(a);
    free(b);
    free(c);

    return 0;
}
```

Calculate $c[i] = a[i] + b[i]$

The same program in CUDA C:

Since our goal is to run this program on the GPU, we must convert the function add into a kernel function, which is a function that runs on the GPU.

A kernel function is called from the CPU to execute on the GPU across many threads simultaneously. It must have the `__global__` specifier and the return type `void`. Instead of using a for loop, each GPU thread computes its own index and processes one element:

```
__global__ void device_add(int *a, int *b, int *c, int n)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < n) {
        c[tid] = a[tid] + b[tid];
    }
}
```

Each thread uses the built-in variables `blockIdx.x`, `blockDim.x`, and `threadIdx.x` to compute a unique index `tid`, which tells the thread which element of the array it is responsible for. The if statement prevents threads from accessing elements beyond the array length.

```

int *h_a, *h_b, *h_c;        // CPU pointers
int *d_a, *d_b, *d_c;        // GPU pointers
int size = N * sizeof(int);

// Allocate on CPU
h_a = (int *)malloc(size);
h_b = (int *)malloc(size);
h_c = (int *)malloc(size);

// Allocate on GPU
cudaMalloc(&d_a, size);
cudaMalloc(&d_b, size);
cudaMalloc(&d_c, size);

```

Once the CPU arrays are initialized, we use `cudaMemcpy()` to copy data to the GPU, launch the kernel, and copy the result back:

```

// Copy input arrays from CPU to GPU
cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

// Launch the kernel with 256 threads per block
int block_size = 256;
int grid_size = N / block_size + 1;
device_add<<<grid_size, block_size>>>(d_a, d_b, d_c, N);

// Copy result from GPU to CPU
cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

```

The <<<grid_size, block_size>>> syntax tells CUDA how many threads to launch on the GPU. In this case, we will use enough "blocks" of 256 threads each to cover all one million elements, and every thread will run the function called device_add, with the GPU pointers and array size as arguments.

Once the kernel function completes and data is copied back, we need to deallocate memory using free() and cudaFree():

```
free(h_a);  
free(h_b);  
free(h_c);
```

```
cudaFree(d_a);  
cudaFree(d_b);  
cudaFree(d_c);
```

```

#include <stdio.h>
#include <stdlib.h>
#include <assert.h>

#define N 1000000

__global__ void device_add(int *a, int *b, int *c, int
n) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < n) {
        c[tid] = a[tid] + b[tid];
    }
}

int main() {
    int *h_a, *h_b, *h_c;
    int *d_a, *d_b, *d_c;
    int size = N * sizeof(int);

    // Allocate memory on the CPU
    h_a = (int *)malloc(size);
    h_b = (int *)malloc(size);
    h_c = (int *)malloc(size);

    // Allocate memory on the GPU
    cudaMalloc(&d_a, size);
    cudaMalloc(&d_b, size);
    cudaMalloc(&d_c, size);

    // Initialize CPU arrays
    for (int i = 0; i < N; i++) {
        h_a[i] = i;
        h_b[i] = i;
    }

    // Copy data from CPU to GPU
    cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

    // Launch the kernel
    int block_size = 256;
    int grid_size = N / block_size + 1;
    device_add<<<grid_size, block_size>>>>(d_a, d_b, d_c, N);

    // Copy result from GPU to CPU
    cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

    // Validate the result
    for (int i = 0; i < N; i++) {
        assert(h_c[i] == (h_a[i] + h_b[i]));
    }
    printf("array sum is correct\n");

    // Free CPU memory
    free(h_a);
    free(h_b);
    free(h_c);

    // Free GPU memory
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);

    return 0;
}

```

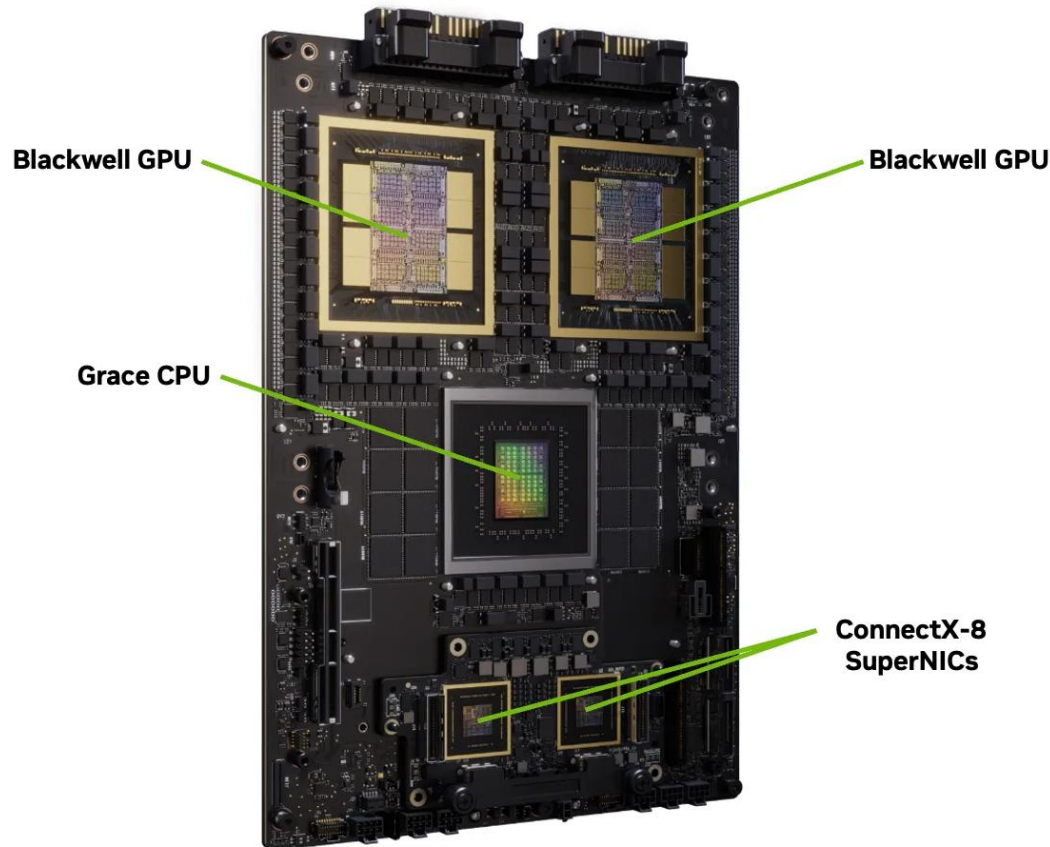
Since no GPU device is available on the login nodes, you must submit the program as a job to run on the GPU compute nodes. Create a batch script called `add_cuda.sh` with the following content:

```
#!/bin/bash
#SBATCH -J add_cuda           # Job name
#SBATCH -o add_cuda.o%j      # Standard output file
#SBATCH -e add_cuda.e%j      # Standard error file
#SBATCH -N 1                  # Number of nodes
#SBATCH -n 1                  # Number of tasks
#SBATCH -p gh-dev             # GPU development queue
#SBATCH -t 00:05:00           # Run time (hh:mm:ss)
#SBATCH -A myproject          # Replace with your allocation name

module load cuda
cd ~/cuda_quickstart
./add_cuda
```


NVIDIA Grace Blackwell Superchip

2 Blackwell GPUs (B100) coupled to each other and to 1 Grace CPU



NVIDIA Grace Blackwell Superchip

Source: [NVIDIA Technical Blog](#)

Inside a Blackwell B200 GPU:

NVIDIA Blackwell ~~Ultra~~ GPU

B200

x16 PCIe Gen 6
256 GB/s CPU Host Interface

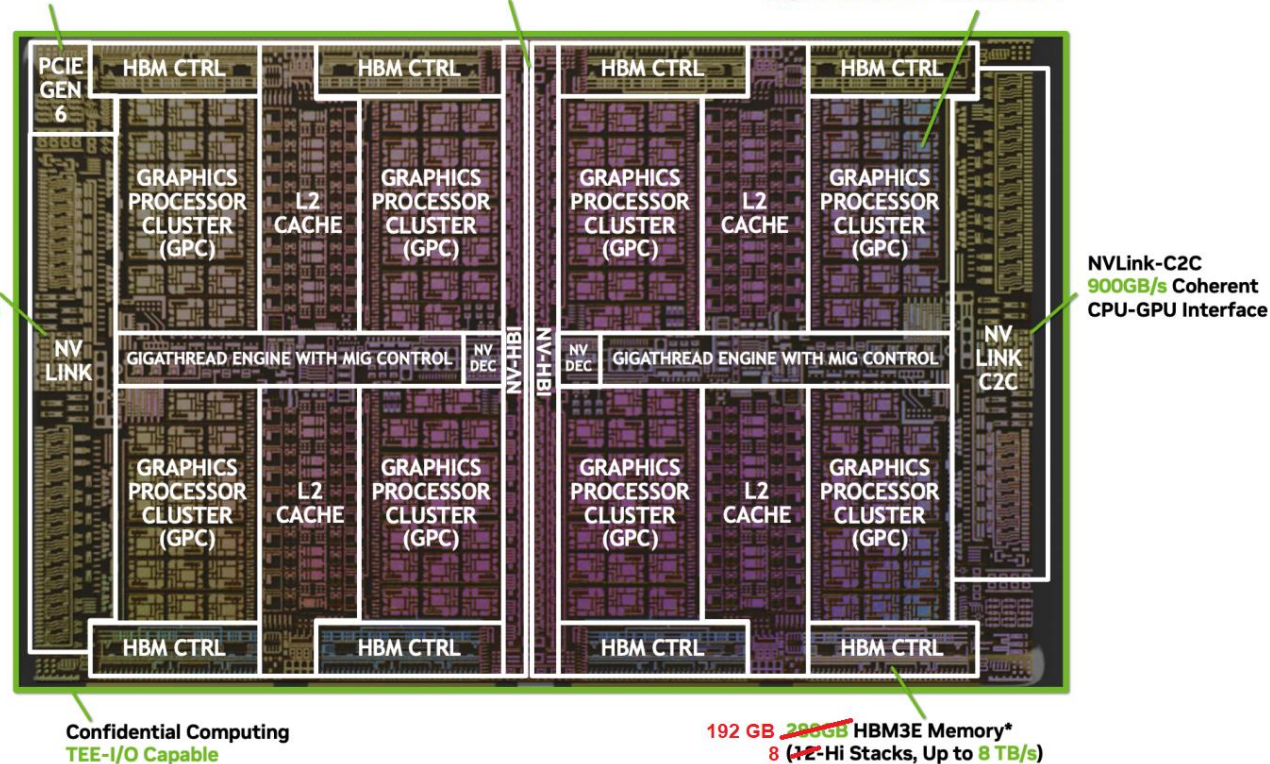
High Bandwidth Interface
10TB/s Die-to-Die

160 SMs per GPU: 640 Tensor Cores*
10 ~~16~~ PetaFLOPS Dense NVFP4

The SMs are paired into Texture Processing Clusters (TPCs), which are in turn grouped into GPU Processing Clusters (GPCs).

NVLink v5
1,800 GB/s to
NVLink Switch

Each of the 8 GPCs contains exactly 20 SMs (in 10 TPCs), for a total of $8 \times 20 = 160$ SMs.



Blackwell GPU chip diagram.

Source: [NVIDIA Developer Blog](#)

An NVIDIA GPU is practically an array of Streaming Multiprocessors (SMs) that are individually and collectively responsible for executing many threads. Each SM contains an assortment of CUDA cores for handling different types of data, including FP32 and FP64. The CUDA cores within an SM are responsible for processing the threads by executing arithmetic and other operations synchronously on warp-sized groups of the various datatypes.

Inside a Blackwell B200 SM:

Each Blackwell B200 SM has:

- 128 FP32 CUDA cores
- 128 INT32 CUDA cores
- 64 FP64 CUDA cores
- 4 Tensor Cores (5th generation)
- 4 Special Function Units
- 4 Texture units

In general, an SM is subdivided into 4 processing blocks. This allows for greater flexibility in scheduling.

The CUDA cores can be scheduled so that on every clock cycle, either 4 separate warps of FP32 or INT32 can be processed, or 2 warps of FP64 (or possibly some combination of these)



10,240 double precision (FP64) CUDA cores (160 SMs x 64 FP64 cuda cores/SM):

One fused multiply-add (FMA) on every cycle. This gives the B200 a peak double precision (FP64) floating-point performance of 40 teraflop/s, computed as follows:

$$10,240 \text{ FP64 CUDA cores} \times 2 \frac{\text{flop}}{\text{core} \cdot \text{cycle}} \times 1.965 \frac{\text{Gcycle}}{\text{s}} \approx 40 \frac{\text{Tflop}}{\text{s}}$$

The factor of 2 flop/core/cycle comes from the ability of each core to execute FMA instructions

20,480 single precision (FP32) CUDA cores (160 SMs x 128 FP32 cuda cores/SM):

Peak FP32 rate it is exactly double the above:

$$20,480 \text{ FP32 CUDA cores} \times 2 \frac{\text{flop}}{\text{core} \cdot \text{cycle}} \times 1.965 \frac{\text{Gcycle}}{\text{s}} \approx 80 \frac{\text{Tflop}}{\text{s}}$$

If we compare the B200's peak FP64 rate to that of the companion Grace processor in the GB200, assuming the latter runs at its maximum "Turbo Boost" frequency on all 72 cores, with four 128-bit vector units per core doing FMAs on every cycle:

$$72 \text{ cores} \times 4 \frac{\text{VPUs}}{\text{core}} \times 2 \frac{\text{FP64-lanes}}{\text{VPU}} \times 2 \frac{\text{flop}}{\text{lane} \cdot \text{cycle}} \times 3.44 \frac{\text{Gcycle}}{\text{s}} \approx 4.0 \frac{\text{Tflop}}{\text{s}}$$

Inside a Blackwell B200 SM:

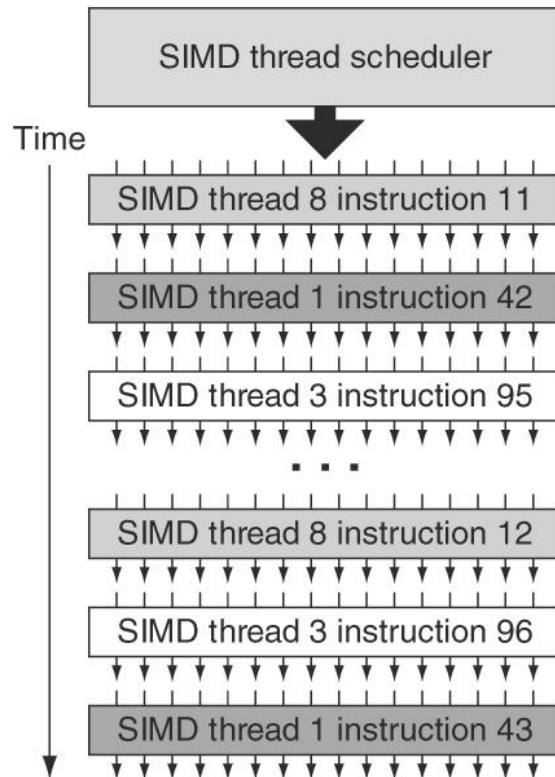
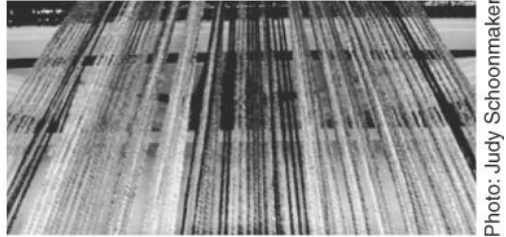
Data are supplied to the main part of this hardware through 32 Load/Store units, as shown at the bottom of each processing block:

- Loads go into the ample 64 KB register file or the 64 KB tensor memory (TMEM) associated with each processing block. A Blackwell B200 SM therefore has, in total, 256 KB of register memory and 256 KB of TMEM.
- Stores, on the other hand, are generally pushed out to the 256 KB L1 data cache. From there, they can propagate to the globally shared L2 cache, then into global memory. Note that the L1 cache size can be adjusted downward to make room for special "shared memory" within the SM, where each data item is accessible to any thread.

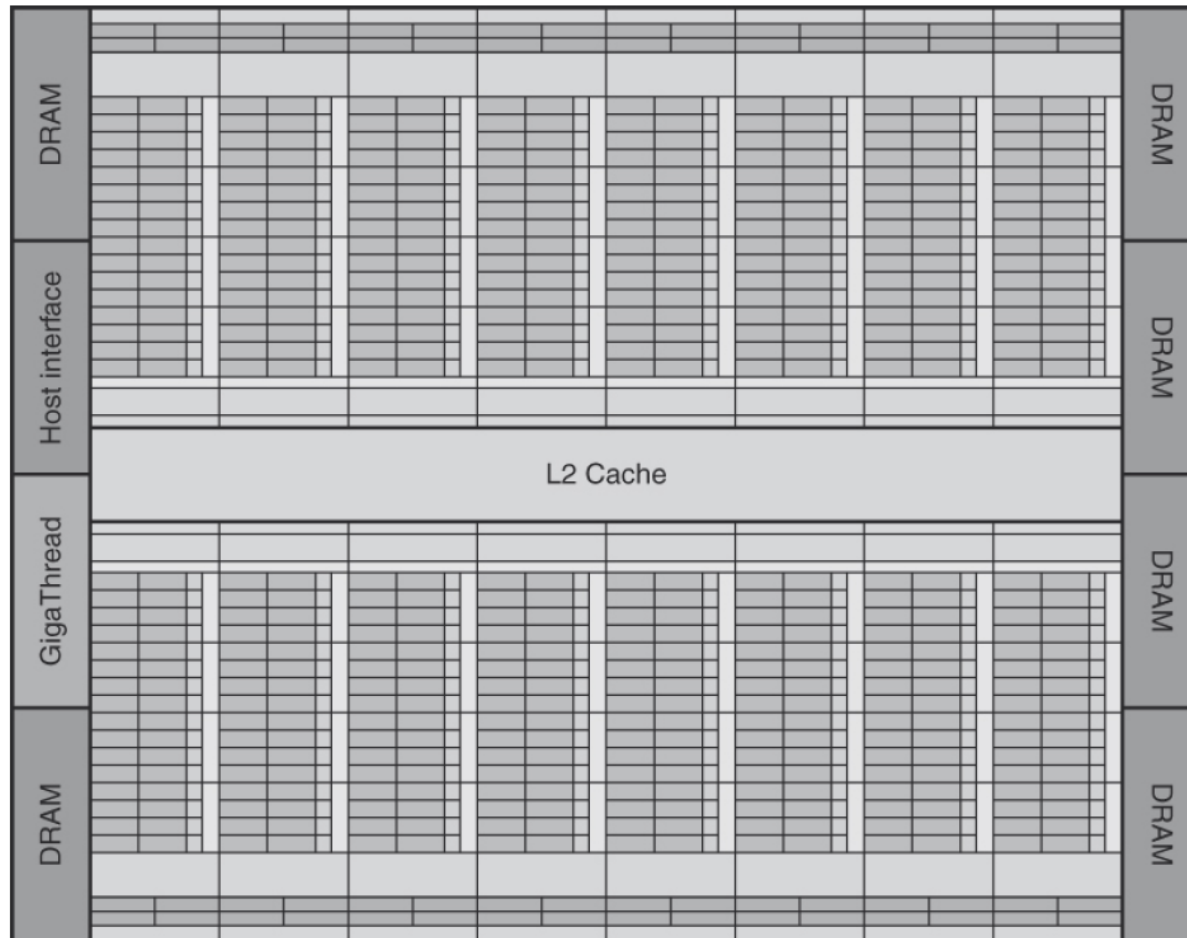
Data are loaded into the tensor cores through the special "tensor memory accelerator" (TMA), which can do bulk asynchronous loads independently, thereby taking pressure off the CUDA cores.



Scheduling of SIMD instructions



The scheduler selects a ready thread of SIMD instructions and issues an instruction synchronously to all the SIMD Lanes executing the SIMD thread. Since threads of SIMD instructions are independent, the scheduler may select a different SIMD thread each time.



I

Figure 4.15 Floor plan of the Fermi GTX 480 GPU. This diagram shows 16 multithreaded SIMD Processors. The Thread Block Scheduler is highlighted on the left. The GTX 480 has 6 GDDR5 ports, each 64 bits wide, supporting up to 6 GB of capacity. The Host Interface is PCI Express 2.0 x 16. Giga Thread is the name of the scheduler that distributes thread blocks to Multiprocessors, each of which has its own SIMD Thread Scheduler.

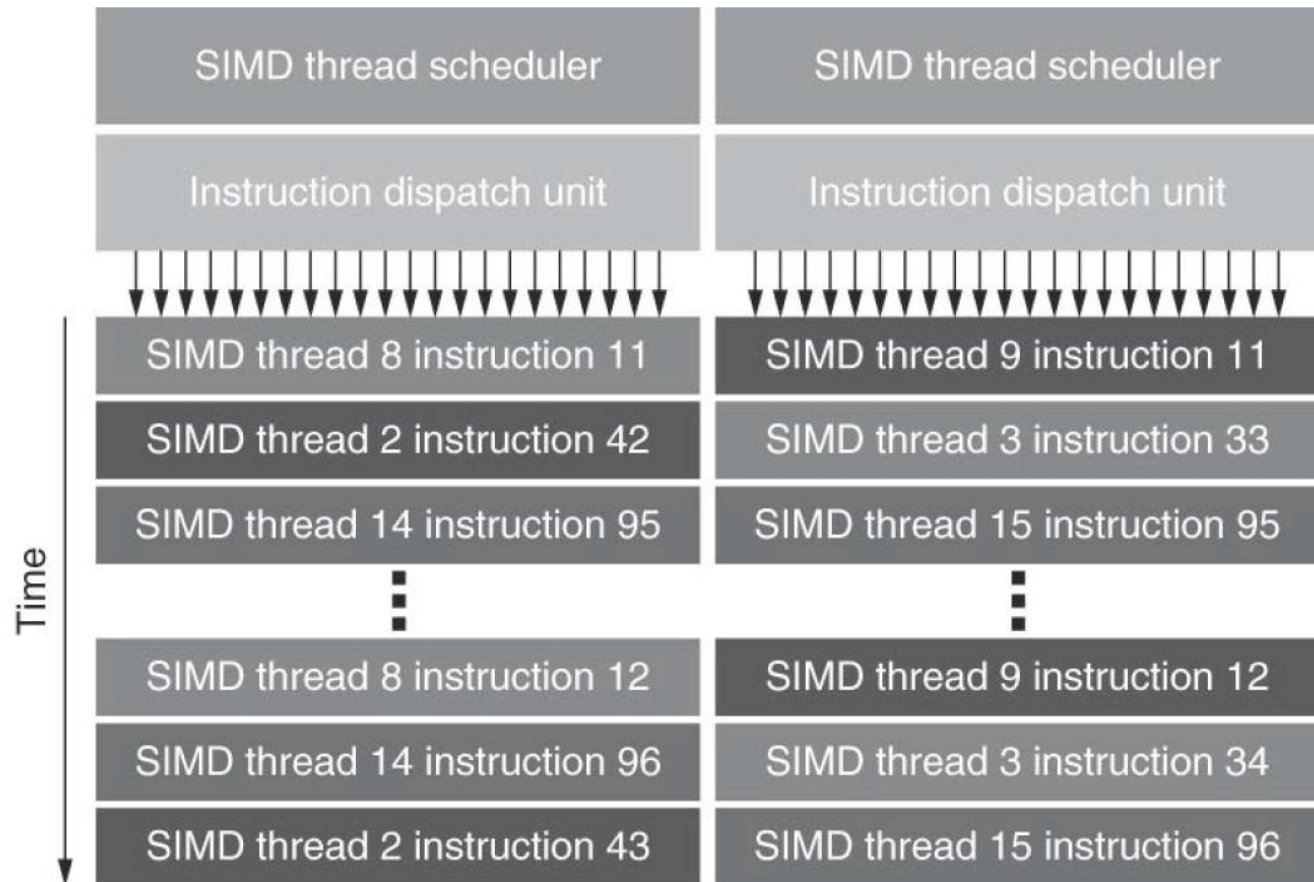
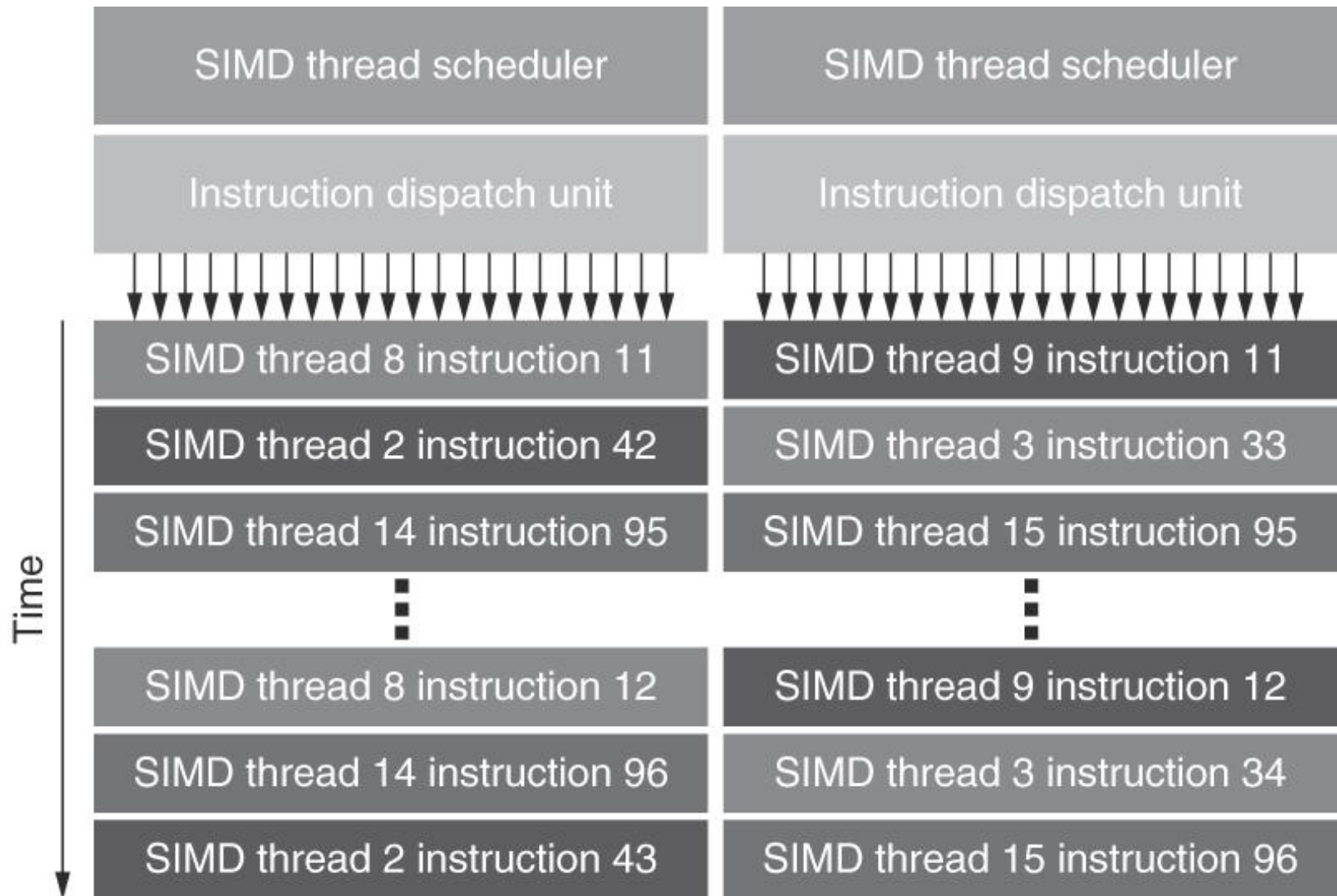


Figure 4.19 Block Diagram of Fermi's Dual SIMD Thread Scheduler. Compare this design to the single SIMD Thread Design in Figure 4.16.



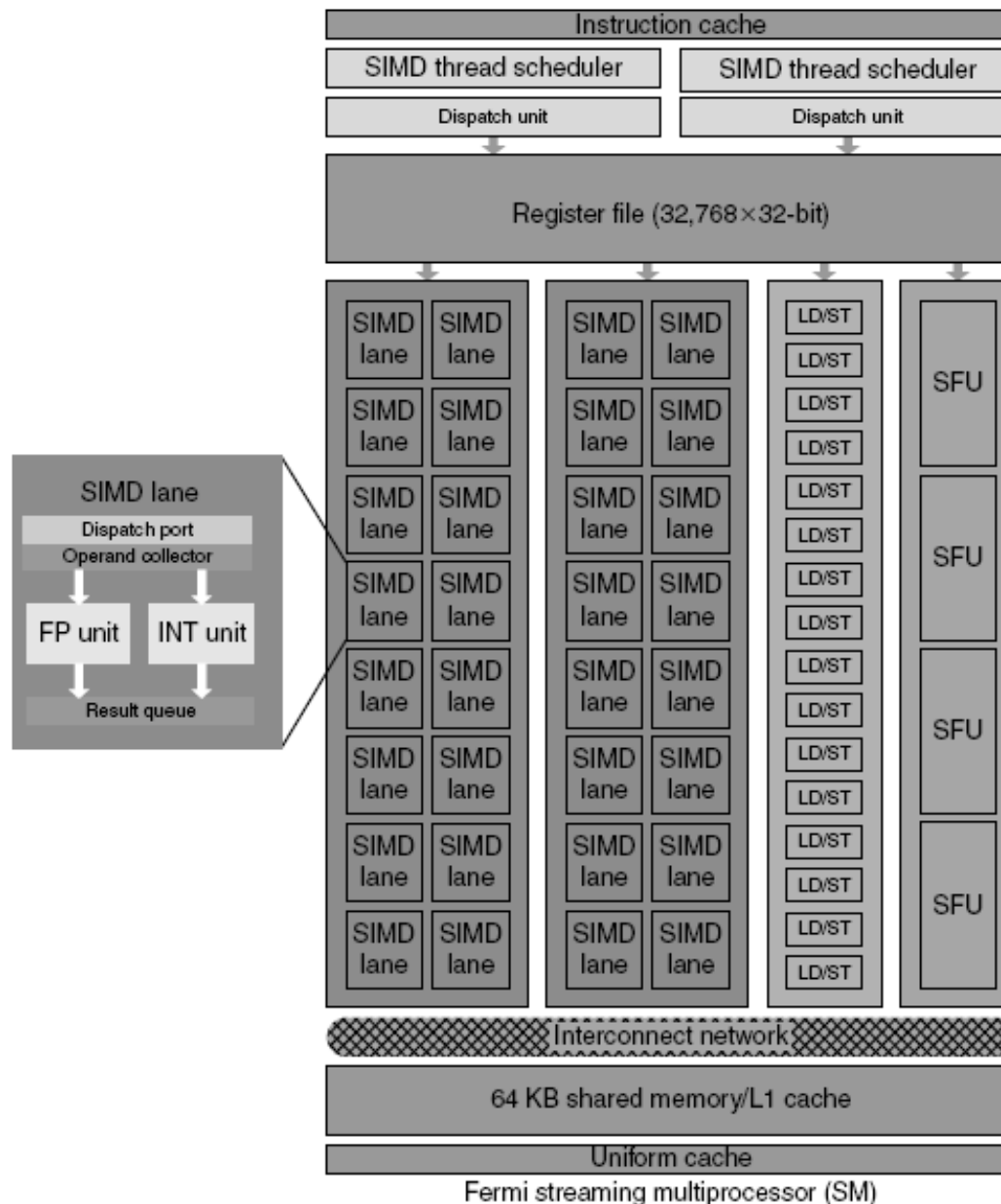
Block Diagram of Fermi's Dual SIMD Thread Scheduler

Compare this design to the single SIMD Thread Design

Fermi Architecture Innovations

- Each SIMD processor has
 - Two SIMD thread schedulers, two instruction dispatch units
 - 16 SIMD lanes (SIMD width=32, chime=2 cycles), 16 load-store units, 4 special function units
 - Thus, two threads of SIMD instructions are scheduled every two clock cycles
- Fast double precision: gen- 78 → 515 GFLOPs for DAXPY
- Caches for GPU memory: I/D L1/SIMD proc and shared L2
- 64-bit addressing and unified address space: C/C++ ptrs
- Error correcting codes: dependability for long-running apps
- Faster context switching: hardware support, 10X faster
- Faster atomic instructions: 5-20X faster than earlier

Fermi Multithreaded SIMD Proc.



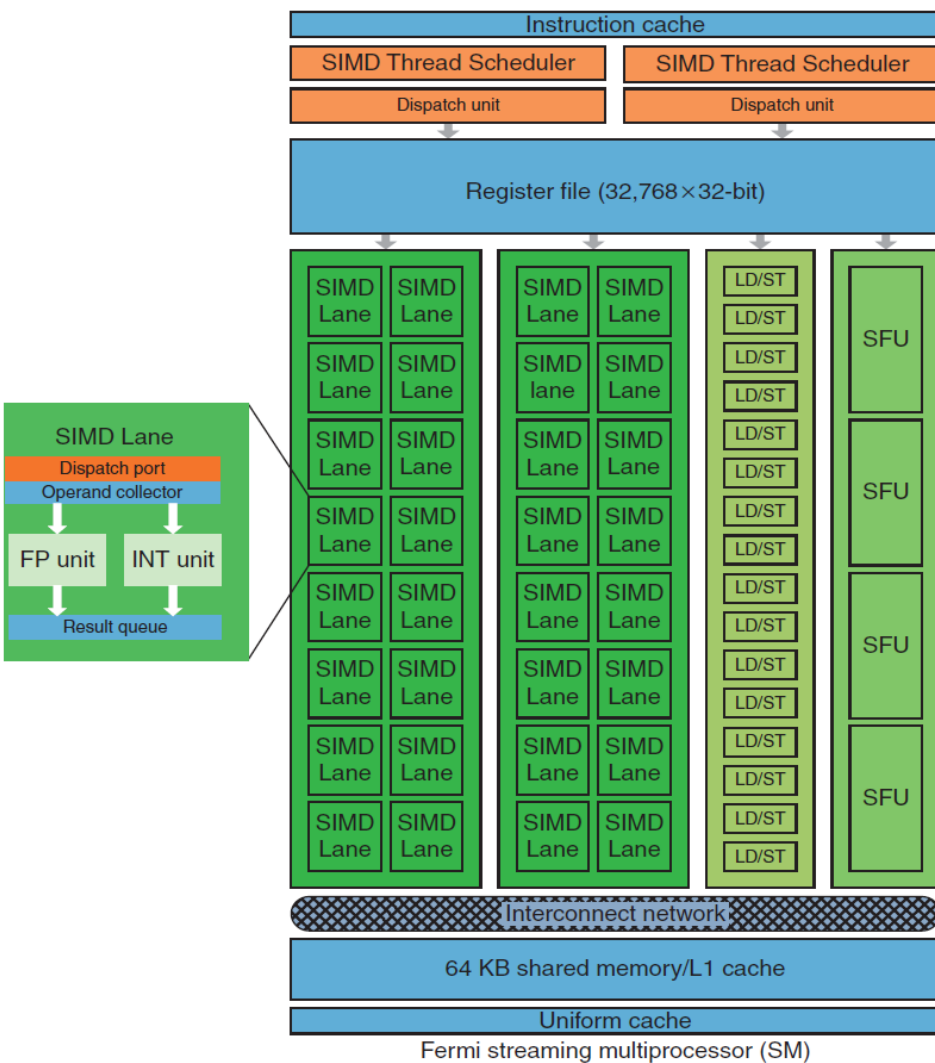
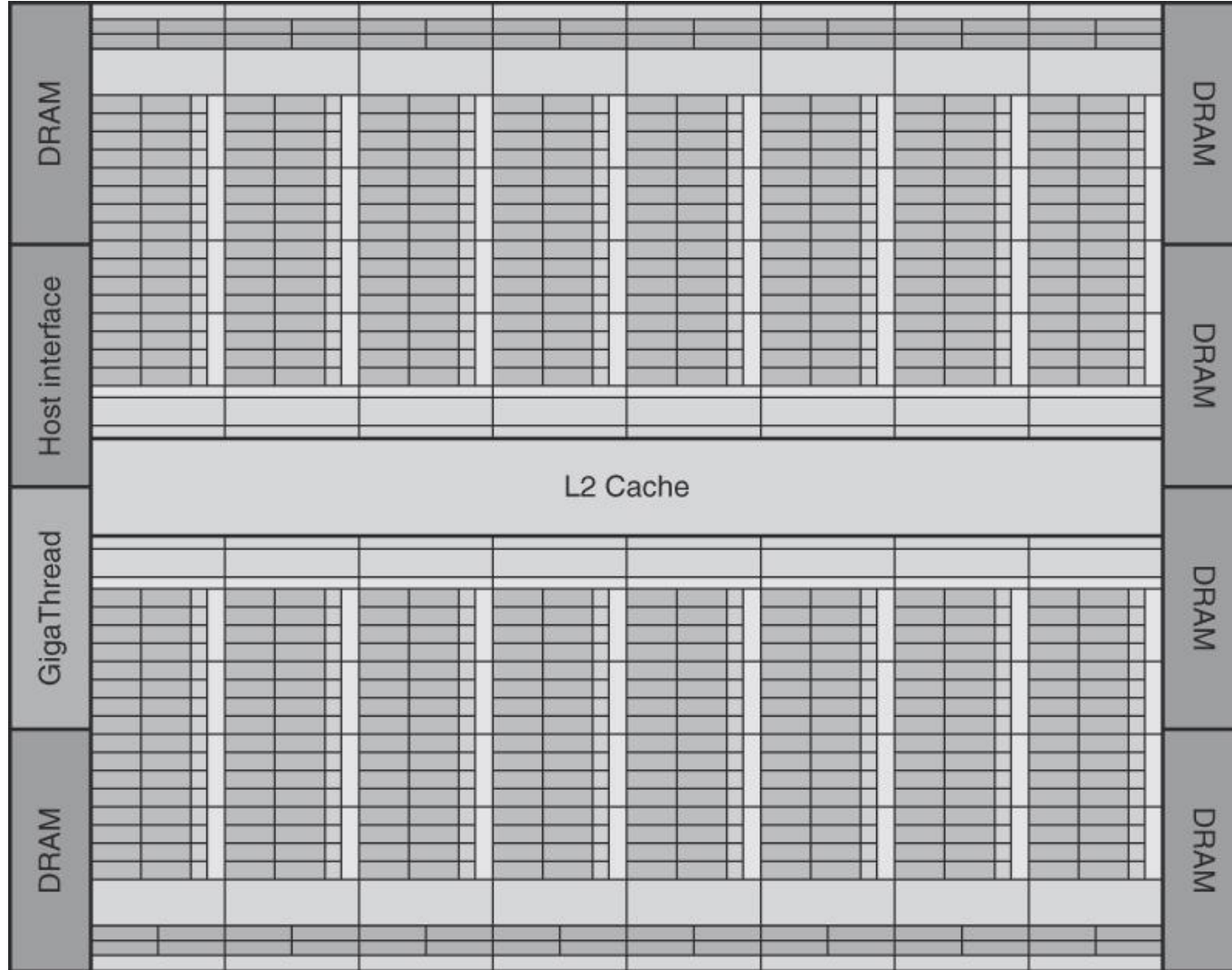


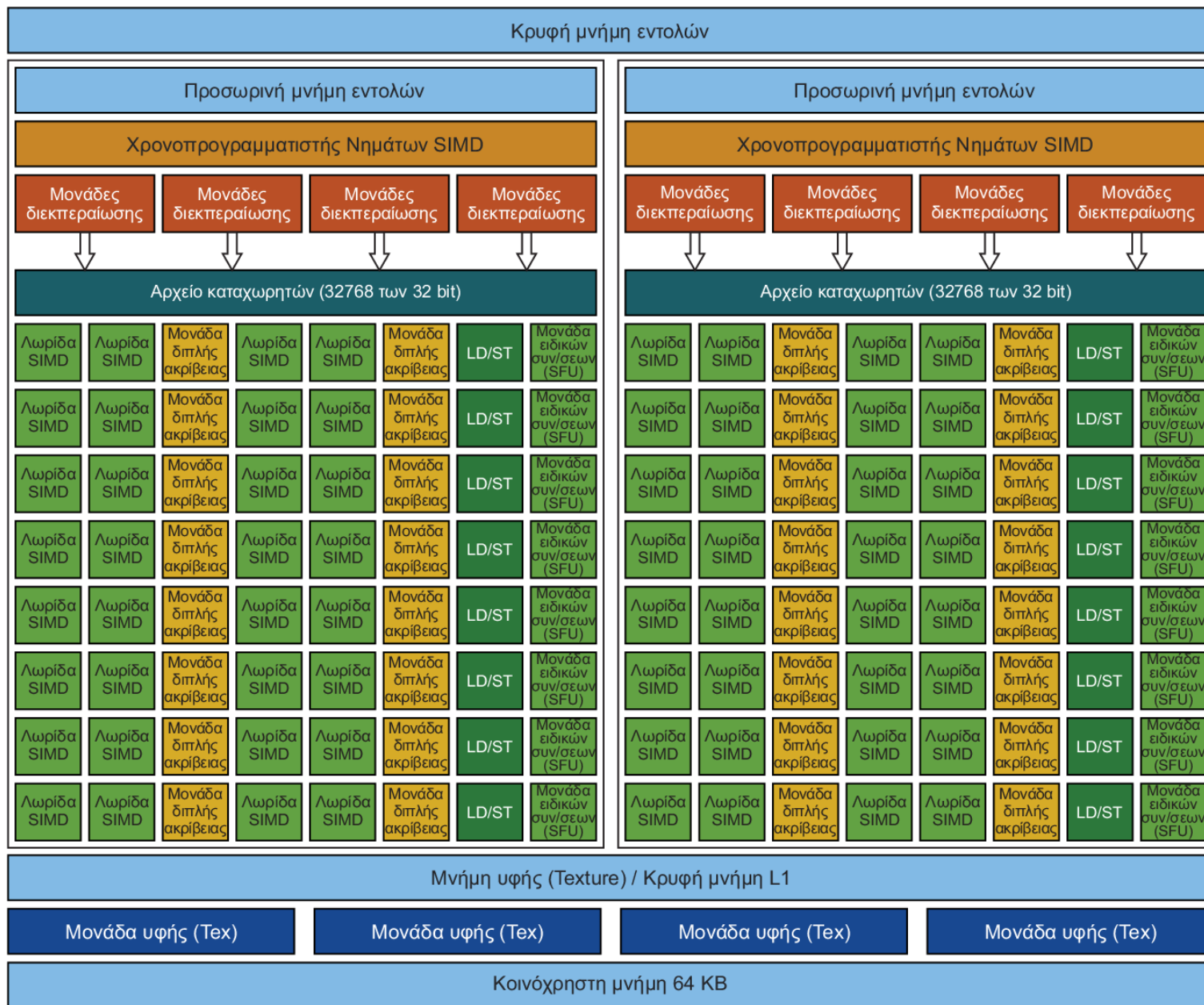
Figure 4.20 Block diagram of the multithreaded SIMD Processor of a Fermi GPU. Each SIMD Lane has a pipelined floating-point unit, a pipelined integer unit, some logic for dispatching instructions and operands to these units, and a queue for holding results. The four Special Function units (SFUs) calculate functions such as square roots, reciprocals, sines, and cosines.

Fermi GTX 480 GPU Floor plan



6 GDDR5 ports, 64 bits wide, up to 6 GB size. Thread Block Scheduler shown on the left

Pascal GTX 1080 GPU Floor Plan



Pascal vs Turing

GPU Features	GTX 1080Ti	RTX 2080 Ti	Quadro P6000	Quadro RTX 6000
Architecture	Pascal	Turing	Pascal	Turing
GPCs	6	6	6	6
TPCs	28	34	30	36
SMs	28	68	30	72
CUDA Cores / SM	128	64	128	64
CUDA Cores / GPU	3584	4352	3840	4608
Tensor Cores / SM	NA	8	NA	8
Tensor Cores / GPU	NA	544	NA	576
RT Cores	NA	68	NA	72
GPU Base Clock MHz (Reference / Founders Edition)	1480 / 1480	1350 / 1350	1506	1455

NVIDIA Instruction Set Arch.

- ISA is an abstraction of the hardware instruction set
 - “Parallel Thread Execution (PTX)”
 - Uses virtual registers
 - Translation to machine code is performed in *software*
 - Example: one CUDA thread, 8192 of these created!

```
shl.s32      R8, blockIdx, 9    ; Thread Block ID * Block size (512 or 29)
add.s32      R8, R8, threadIdx ; R8 = i = my CUDA thread ID
ld.global.f64 RD0, [X+R8]      ; RD0 = X[i]
ld.global.f64 RD2, [Y+R8]      ; RD2 = Y[i]
mul.f64 R0D, RD0, RD4           ; Product in RD0 = RD0 * RD4 (scalar a)
add.f64 R0D, RD0, RD2           ; Sum in RD0 = RD0 + RD2 (Y[i])
st.global.f64 [Y+R8], RD0       ; Y[i] = sum (X[i]*a + Y[i])
```


The format of a PTX instruction is

`opcode.type d, a, b, c;`

where `d` is the destination operand; `a`, `b`, and `c` are source operands; and the operation type is one of the following:

Type	.type Specifier
Untyped bits 8, 16, 32, and 64 bits	.b8, .b16, .b32, .b64
Unsigned integer 8, 16, 32, and 64 bits	.u8, .u16, .u32, .u64
Signed integer 8, 16, 32, and 64 bits	.s8, .s16, .s32, .s64
Floating Point 16, 32, and 64 bits	.f16, .f32, .f64

Source operands are 32-bit or 64-bit registers or a constant value. Destinations are registers, except for store instructions.

```
// Invoke DAXPY
daxpy(n, 2.0, x, y);
// DAXPY in C
void daxpy(int n, double a, double *x, double *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = a*x[i] + y[i];
}
```

```
// Invoke DAXPY with 256 threads per Thread Block
__host__
int nblocks = (n+ 255) / 256;
    daxpy<<<nblocks, 256>>>(n, 2.0, x, y);
// DAXPY in CUDA
__device__
void daxpy(int n, double a, double *x, double *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}
```

```
shl.u32 R8, blockIdx, 9    ; Thread Block ID * Block size (512 or 29)
add.u32 R8, R8, threadIdx ; R8 = i = my CUDA Thread ID
shl.u32 R8, R8, 3          ; byte offset
ld.global.f64 RD0, [X+R8] ; RD0 = X[i]
ld.global.f64 RD2, [Y+R8] ; RD2 = Y[i]
mul.f64 RD0, RD0, RD4      ; Product in RD0 = RD0 * RD4 (scalar a)
add.f64 RD0, RD0, RD2      ; Sum in RD0 = RD0 + RD2 (Y[i])
st.global.f64 [Y+R8], RD0 ; Y[i] = sum (X[i]*a + Y[i])
```

Conditional Branching

- Like vector architectures, GPU branch hardware uses internal masks
- Also uses
 - Branch synchronization stack
 - Entries consist of masks for each SIMD lane
 - I.e. which threads commit their results (all threads execute)
 - Instruction markers to manage when a branch diverges into multiple execution paths
 - Push on divergent branch
 - ...and when paths converge
 - Act as barriers
 - Pops stack
- Per-thread-lane 1-bit predicate register, specified by programmer

Example

```
if (X[i] != 0)
    X[i] = X[i] - Y[i];
else X[i] = Z[i];
```

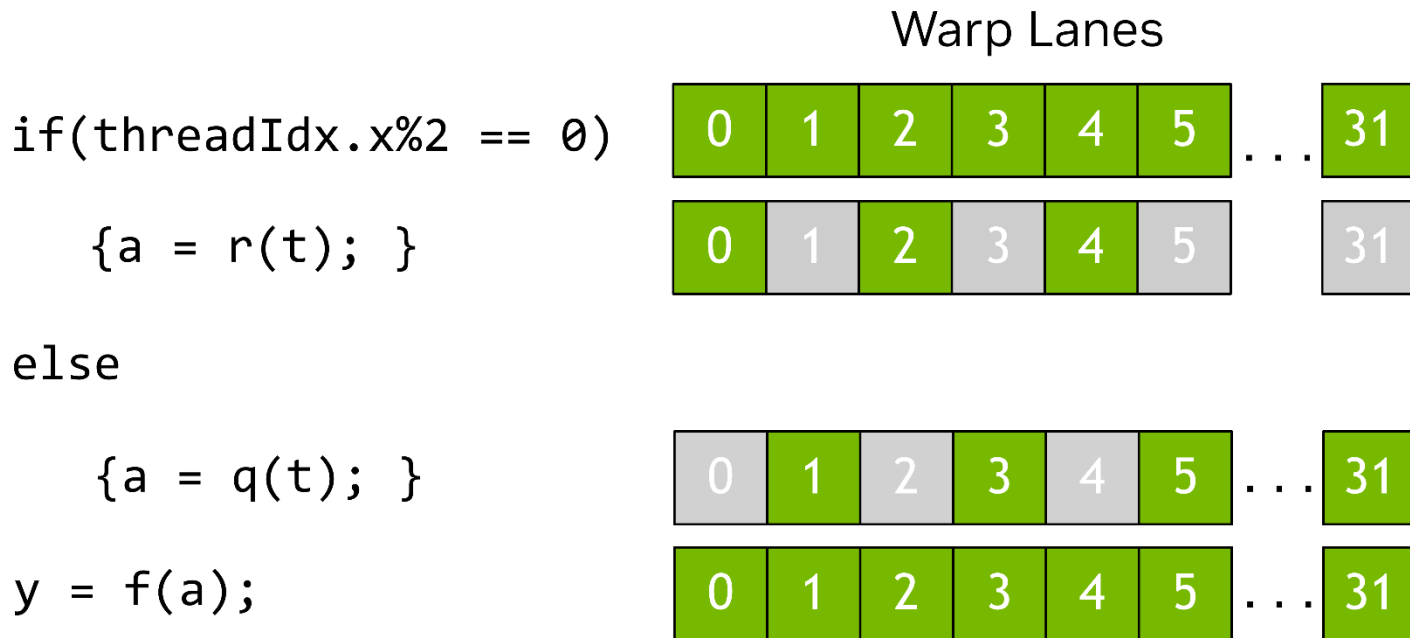
```
ld.global.f64    RD0, [X+R8]          ; RD0 = X[i]
setp.neq.s32     P1, RD0, #0          ; P1 is predicate register 1
@!P1, bra        ELSE1, *Push         ; Push old mask, set new mask bits
                                           ; if P1 false, go to ELSE1

ld.global.f64    RD2, [Y+R8]          ; RD2 = Y[i]
sub.f64          RD0, RD0, RD2         ; Difference in RD0
st.global.f64    [X+R8], RD0          ; X[i] = RD0
@P1, bra        ENDIF1, *Comp         ; complement mask bits
                                           ; if P1 true, go to ENDIF1

ELSE1:           ld.global.f64 RD0, [Z+R8] ; RD0 = Z[i]
                  st.global.f64 [X+R8], RD0 ; X[i] = RD0

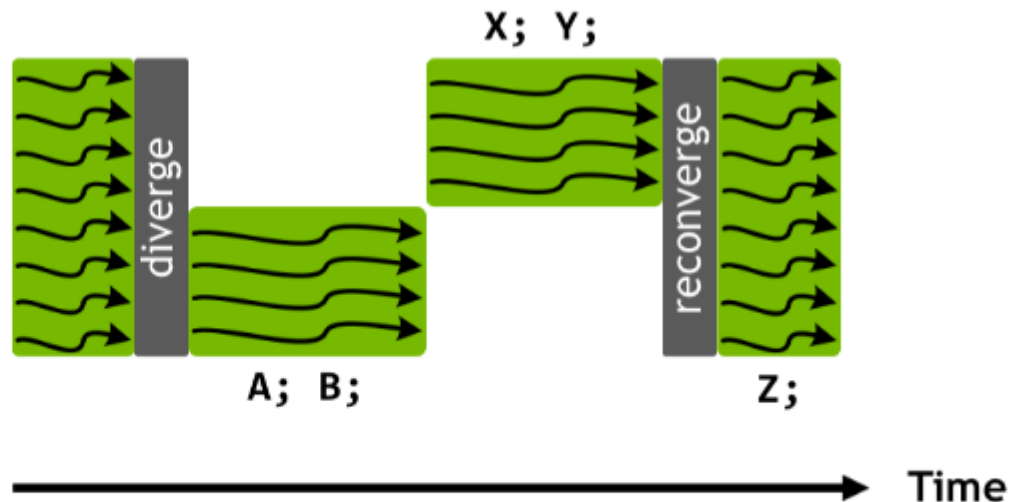
ENDIF1: <next instruction>, *Pop      ; pop to restore old mask
```

Branches and warp threads



SIMT warp execution model before Volta architecture

```
if (threadIdx.x < 4) {  
    A;  
    B;  
} else {  
    X;  
    Y;  
}  
Z;
```



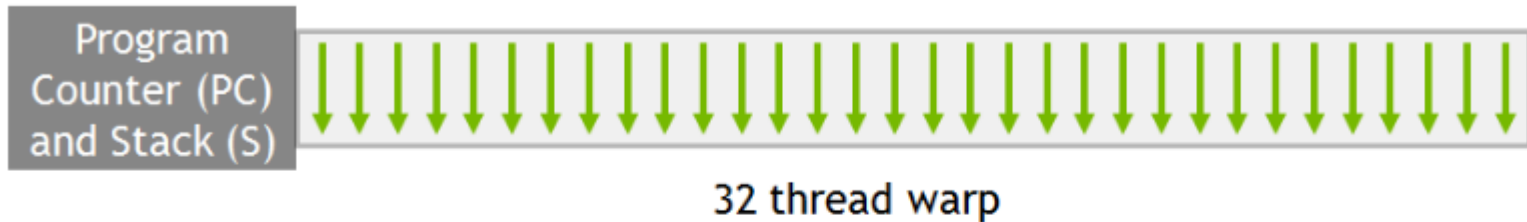
Thread scheduling under the SIMT warp execution model of Pascal and earlier NVIDIA GPUs. Capital letters represent statements in the program pseudocode. Divergent branches within a warp are serialized so that all statements in one side of the branch are executed together to completion before any statements in the other side are executed. After the else statement, the threads of the warp will typically reconverge.

Independent Thread Scheduling

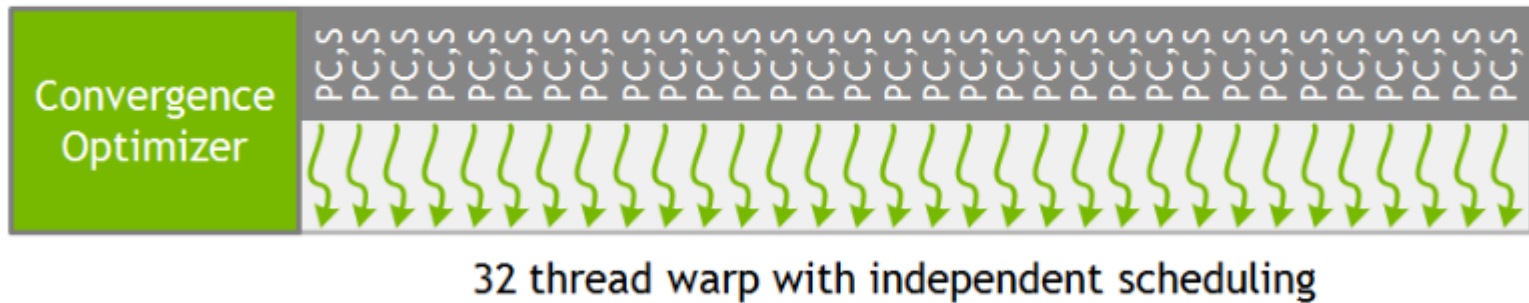
- Before Independent Thread Scheduling (ITS), a warp shared a single Program Counter (PC) and a single active execution mask stack.
- If threads diverged, the hardware rigidly serialized the paths. Thread A had to wait for Thread B to finish its entire divergent branch before they could talk to each other again. This led to serious issues like execution deadlocks and underutilized hardware.
- ITS introduced in Volta (Aug. 2017)
- every individual thread its own independent Program Counter and execution state, while still sharing the same physical instruction issue pipeline.

independent thread scheduling (Volta architecture and forward)

Pre-Volta



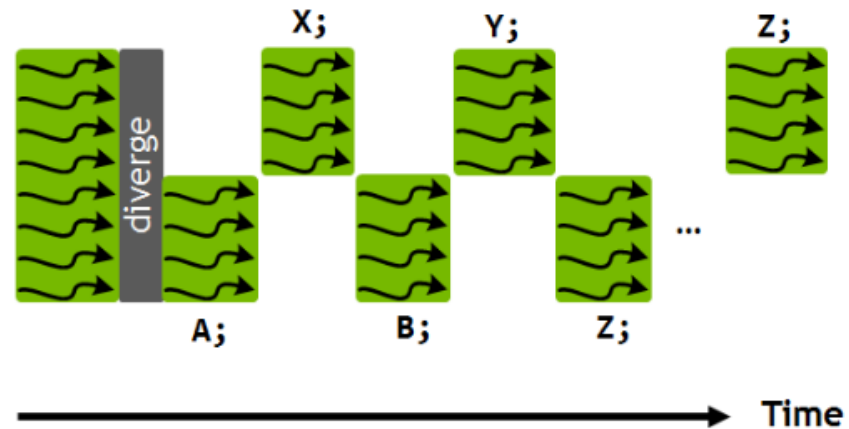
Volta



Volta (bottom) independent thread scheduling architecture block diagram compared to Pascal and earlier architectures (top). Volta maintains per-thread scheduling resources such as program counter (PC) and call stack (S), while earlier architectures maintained these resources per warp.

independent thread scheduling (Volta architecture and forward)

```
if (threadIdx.x < 4) {  
    A;  
    B;  
} else {  
    X;  
    Y;  
}  
Z;
```

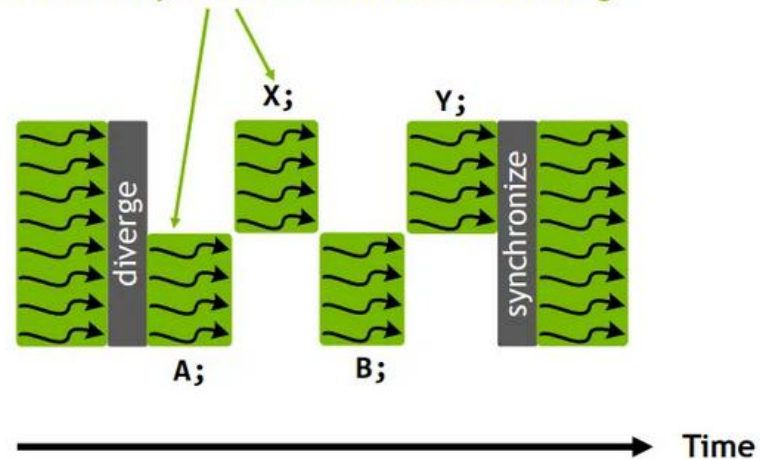


Volta independent thread scheduling enables interleaved execution of statements from divergent branches. This enables execution of fine-grain parallel algorithms where threads within a warp may synchronize and communicate.

VOLTA WARP EXECUTION MODEL

Synchronization may lead to interleaved scheduling!

```
if (threadIdx.x < 4) {  
    A;  
    __syncwarp();  
    B;  
} else {  
    X;  
    __syncwarp();  
    Y;  
}  
__syncwarp();
```



Software synchronization also supported, e.g. locks for doubly-linked list!

```

if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
__syncwarp()

```

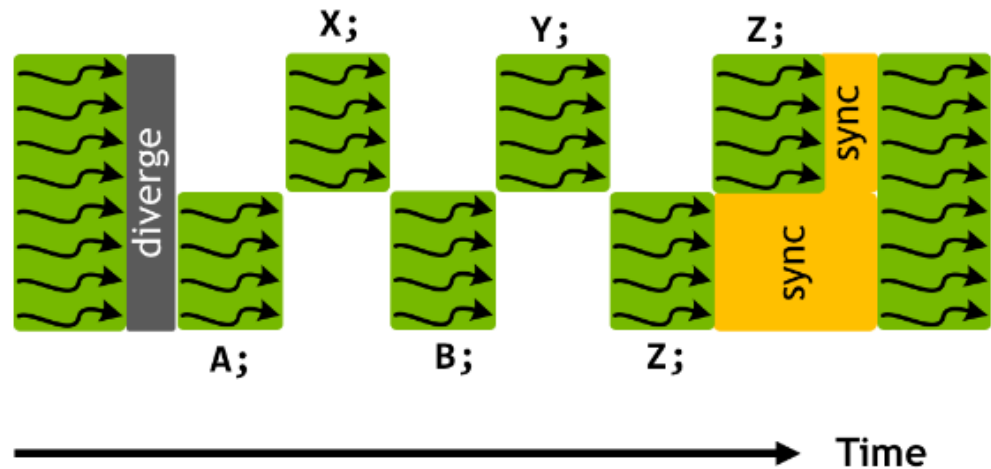


Figure 23. Programs use Explicit Synchronization to Reconverge Threads in a Warp

Helper slides for ITS

1. Eliminating Starvation and Intra-Warp Deadlocks

Before ITS, lock-free algorithms that relied on threads within the same warp communicating via spinning locks or semaphores could cause an infinite loop/deadlock.

The Problem (Pre-Volta):

Imagine Thread 0 locks a resource, and Thread 1 spins in a `while(!locked)` loop waiting for Thread 0 to release it. Because they share a single PC, the hardware might lock the warp into executing Thread 1's spin-loop indefinitely. Thread 0 never gets a turn to run its path and release the lock.

The ITS Solution:

With ITS, the scheduler is not forced to finish one side of a branch before starting the other. It can dynamically execute a few instructions from Thread 1 (the spin loop), realize it's blocked, and immediately switch the execution path within the same warp to let Thread 0 run a few instructions to release the lock. This makes fine-grained, lock-free intra-warp data structures completely safe.

2. Smarter Sub-Warp Yielding and Latency Hiding

In older architectures, if a single thread inside a warp hit a long-latency operation (like a divergent global memory load), the entire warp (including the threads that didn't need that memory) was stalled and blocked by the scoreboard.

An example with a lock and a warp

```
__global__ void warpLockExample(int* global_data) {
    // A single shared variable for the whole thread block
    __shared__ int lock_var;

    if (threadIdx.x == 0) {
        lock_var = 0; // Initialize to unlocked
    }
    __syncthreads(); // Wait for initialization

    // Every thread tries to grab the lock.
    // atomicExch(address, 1) sets the lock to 1 and returns the OLD value.
    // If the old value was 0, this thread successfully acquired the lock!
    bool is_owner = false;

    while (!is_owner) {
        // Try to acquire the lock
        if (atomicExch(&lock_var, 1) == 0) {
            is_owner = true; // Thread X now owns the lock!
        }

        // --- THE SPIN LOCK ---
        // If atomicExch returned 1, is_owner stays false,
        // and the thread loops back to try again.
    }

    // CRITICAL SECTION (Only one thread executes this at a time)
    global_data[threadIdx.x] += 10;

    // RELEASE THE LOCK
    atomicExch(&lock_var, 0);
}
```

The Pre-Volta Deadlock - No independent thread scheduling

Before ITS, a warp had only one Program Counter (PC). Let's see what happens when the 32 threads of Warp 0 execute this code on an older architecture (like Pascal):

The Race: All 32 threads execute `atomicExch(&lock_var, 1)` at the exact same cycle. Let's say Thread 0 wins the hardware arbitration. Its `is_owner` becomes true. For Threads 1 through 31, `is_owner` remains false.

The Branch Divergence: The warp hits the `while(!is_owner)` loop.

Thread 0 wants to exit the loop (evaluate to false) and move to the critical section.

Threads 1-31 want to stay in the loop (evaluate to true).

The Deadlock: Because there is only one PC, the hardware must pick one path to execute first. It prioritizes the active while loop body for Threads 1-31. The hardware masks off Thread 0 (deactivates its lane) and forces Threads 1-31 to spin.

Why it never ends: Threads 1-31 execute `atomicExch(&lock_var, 1)`. Because Thread 0 never got the chance to advance its PC and reach the release code (`atomicExch(&lock_var, 0)`), `lock_var` stays 1 forever. Threads 1-31 spin infinitely. Thread 0 is starved. The GPU hangs.

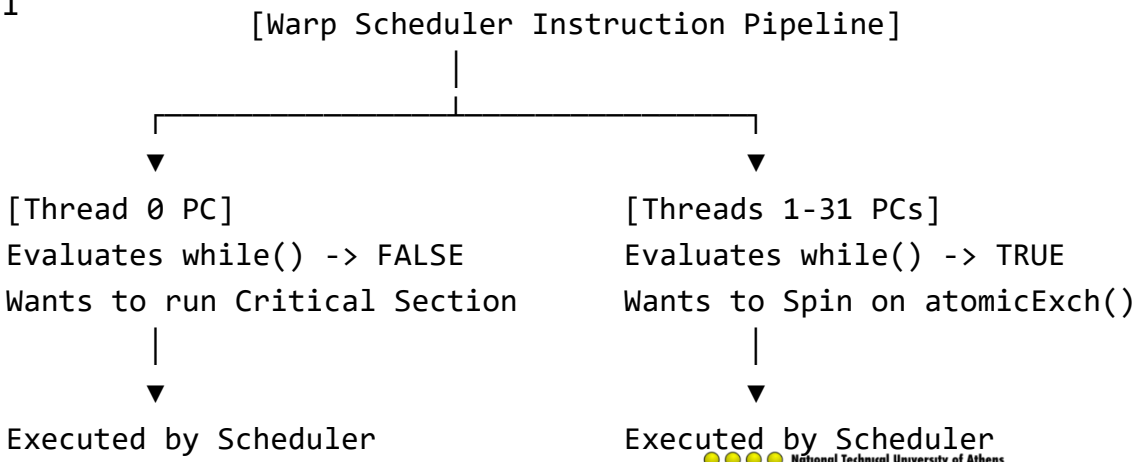
- The Race:** Thread 0 wins the lock. lock_var becomes 1. Threads 1-31 fail.
- The Divergence:** Threads 1-31 enter the loop body to spin. Thread 0 moves past the loop to the Critical Section.
- Interleaved Execution:** The warp scheduler sees that the warp has divergent PCs. It doesn't lock up. It uses its internal scheduling policy to slice time:
- Cycle 1:** It issues instructions for Threads 1-31. They run atomicExch, fail (receive 1), and loop.
- Cycle 2:** Instead of staying stuck there, the scheduler switches to **Thread 0's PC** within the same warp. Thread 0 executes its critical section math.
- Cycle 3:** Thread 0 reaches the release statement: atomicExch(&lock_var, 0).
- The State Change:** Thread 0 writes a 0 back into the shared lock_var variable.
- The Next Winner:** On a subsequent cycle, the scheduler executes the loop for Threads 1-31 again. This time, when they call atomicExch(&lock_var, 1), one of them (say, Thread 1) reads the 0 left by Thread 0.
- Thread 1's is_owner becomes true, its independent PC breaks out of the loop, and the chain continues safely until all 32 threads complete their work.

```
while (!is_owner) {
    // Try to acquire the lock
    if (atomicExch(&lock_var, 1) == 0) {
        is_owner = true; // Thread X now owns the lock!
    }

    // --- THE SPIN LOCK ---
    // If atomicExch returned 1, is_owner stays false,
    // and the thread loops back to try again.
}

// CRITICAL SECTION (Only one thread executes this at a
time)
global_data[threadIdx.x] += 10;

// RELEASE THE LOCK
atomicExch(&lock_var, 0);
```

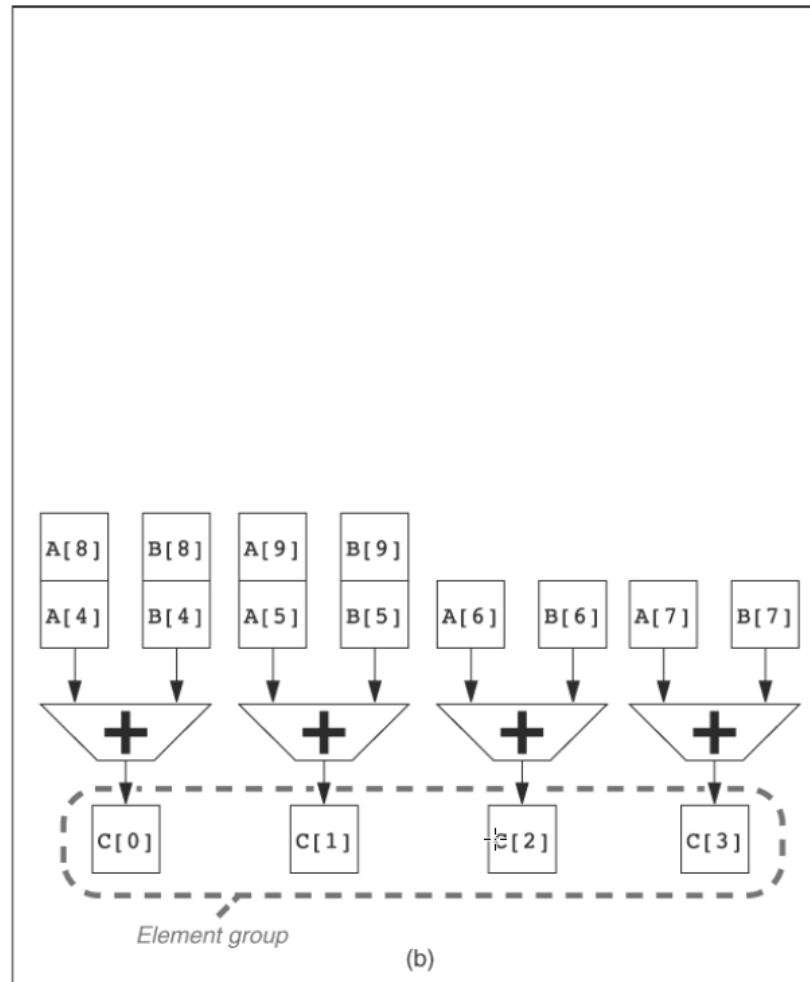
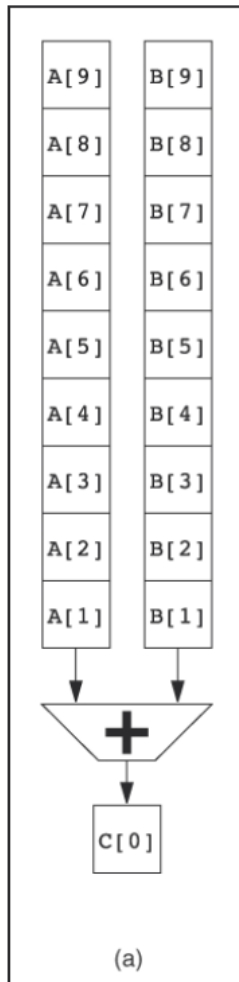


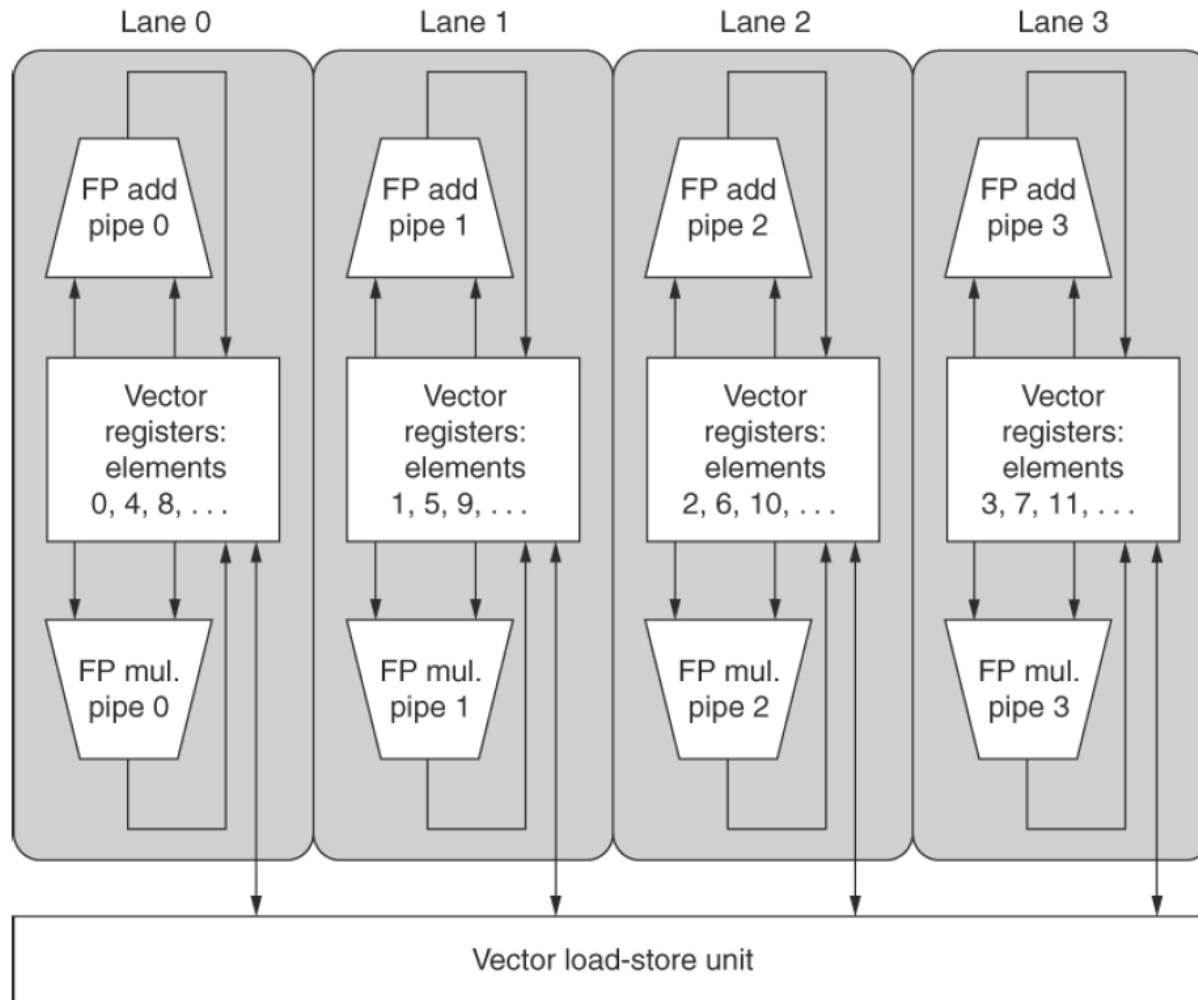
Independent Thread Scheduling:

the warp scheduler maps the single pipeline to two distinct, alternating PC flows

Cycle Cluster	Thread 0 PC Location	Threads 1–31 PC Location	State of lock_var in Shared Memory
Cycle 1	\$L_SPIN_LOOP (Executes exchange)	\$L_SPIN_LOOP (Executes exchange)	0 $\rightarrow$ Turns to 1. Thread 0 wins.
Cycle 2	Advances past bra to Critical Section	Trapped. Jumps back to \$L_SPIN_LOOP	1 (Locked by Thread 0)
Cycle 3	Executes global load/add	Executes \$L_SPIN_LOOP again (Fails)	1 (Threads 1–31 read 1 and stay trapped)
Cycle 4	Advances to Lock Release (atom... [lock_var], 0)	Trapped at \$L_SPIN_LOOP	1 $\rightarrow$ Flips to 0 via Thread 0's write.
Cycle 5	Reaches ret (Exits Kernel)	Executes \$L_SPIN_LOOP again	0 $\rightarrow$ Turns to 1. Next thread wins.

Similarities and differences between vector architectures and GPUs





Type	Vector term	Closest CUDA/NVIDIA GPU term	Comment
Program abstractions	Vectorized Loop	Grid	Concepts are similar, with the GPU using the less descriptive term.
	Chime	--	Since a vector instruction (PTX Instruction) takes just two cycles on Fermi and four cycles on Tesla to complete, a chime is short in GPUs.
Machine objects	Vector Instruction	PTX Instruction	A PTX instruction of a SIMD thread is broadcast to all SIMD Lanes, so it is similar to a vector instruction.
	Gather/Scatter	Global load/store (ld.global/st.global)	All GPU loads and stores are gather and scatter, in that each SIMD Lane sends a unique address. It's up to the GPU Coalescing Unit to get unit-stride performance when addresses from the SIMD Lanes allow it.
	Mask Registers	Predicate Registers and Internal Mask Registers	Vector mask registers are explicitly part of the architectural state, while GPU mask registers are internal to the hardware. The GPU conditional hardware adds a new feature beyond predicate registers to manage masks dynamically.

Processing and memory hardware	Vector Processor	Multithreaded SIMD Processor	These are similar, but SIMD Processors tend to have many lanes, taking a few clock cycles per lane to complete a vector, while vector architectures have few lanes and take many cycles to complete a vector. They are also multithreaded where vectors usually are not.
	Control Processor	Thread Block Scheduler	The closest is the Thread Block Scheduler that assigns Thread Blocks to a multithreaded SIMD Processor. But GPUs have no scalar-vector operations and no unit-stride or strided data transfer instructions, which Control Processors often provide.
	Scalar Processor	System Processor	Because of the lack of shared memory and the high latency to communicate over a PCI bus (1000s of clock cycles), the system processor in a GPU rarely takes on the same tasks that a scalar processor does in a vector architecture.
	Vector Lane	SIMD Lane	Both are essentially functional units with registers.
	Vector Registers	SIMD Lane Registers	The equivalent of a vector register is the same register in all 32 SIMD Lanes of a multithreaded SIMD Processor running a thread of SIMD instructions. The number of registers per SIMD thread is flexible, but the maximum is 64, so the maximum number of vector registers is 64.
	Main Memory	GPU Memory	Memory for GPU versus System memory in vector case.

Figure 4.21 GPU equivalent to vector terms.

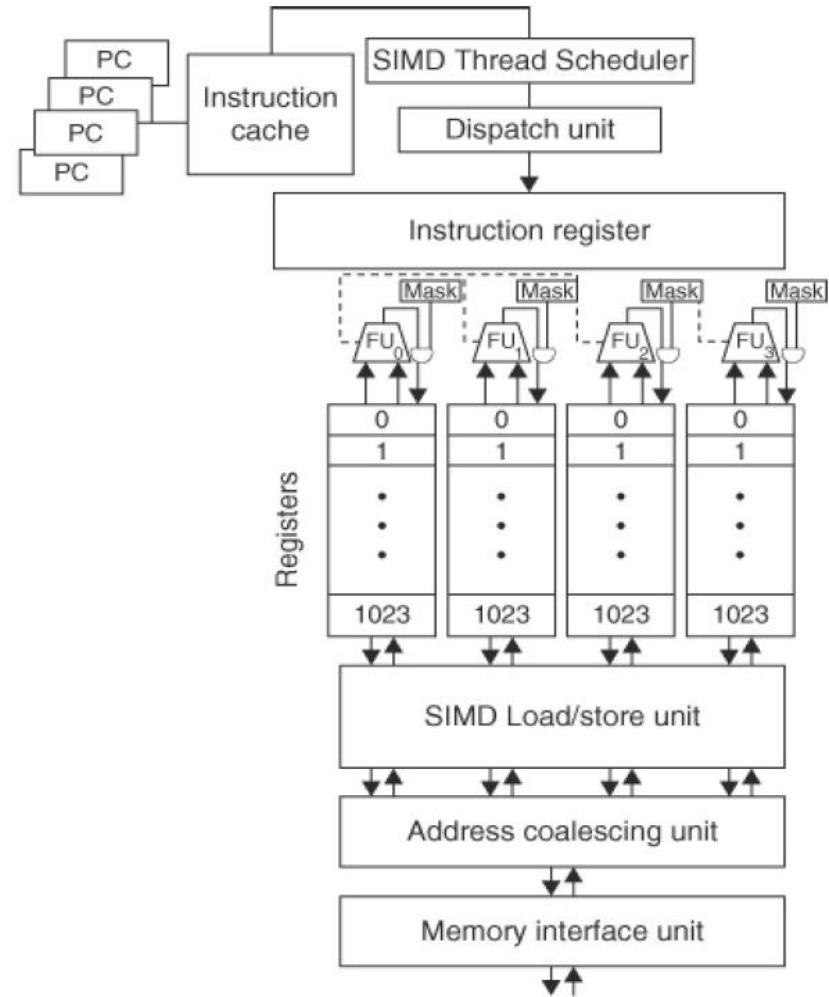
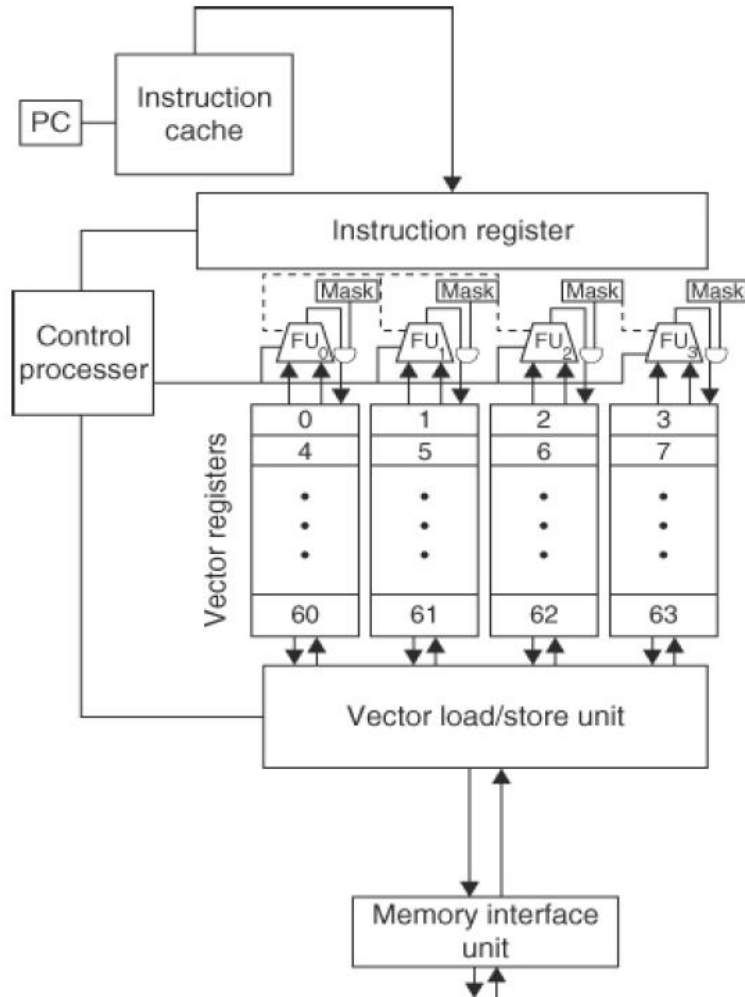


Figure 4.14 Simplified block diagram of a Multithreaded SIMD Processor. It has 16 SIMD lanes. The SIMD Thread Scheduler has, say, 48 independent threads of SIMD instructions that it schedules with a table of 48 PCs.

Feature	Multicore with SIMD	GPU
SIMD processors	4 to 8	8 to 16
SIMD lanes/processor	2 to 4	8 to 16
Multithreading hardware support for SIMD threads	2 to 4	16 to 32
Typical ratio of single-precision to double-precision performance	2:1	2:1
Largest cache size	8 MB	0.75 MB
Size of memory address	64-bit	64-bit
Size of main memory	8 GB to 256 GB	4 to 6 GB
Memory protection at level of page	Yes	Yes
Demand paging	Yes	No
Integrated scalar processor/SIMD processor	Yes	No
Cache coherent	Yes	No

Figure 4.23 Similarities and differences between multicore with Multimedia SIMD extensions and recent GPUs.

Type	More descriptive name used in this book	Official CUDA/NVIDIA term	Book definition and AMD and OpenCL terms	Official CUDA/NVIDIA definition
Program abstractions	Vectorizable loop	Grid	A vectorizable loop, executed on the GPU, made up of one or more “Thread Blocks” (or bodies of vectorized loop) that can execute in parallel. OpenCL name is “index range.” AMD name is “NDRange”.	A grid is an array of thread blocks that can execute concurrently, sequentially, or a mixture.
	Body of Vectorized loop	Thread Block	A vectorized loop executed on a multithreaded SIMD Processor, made up of one or more threads of SIMD instructions. These SIMD Threads can communicate via Local Memory. AMD and OpenCL name is “work group”.	A thread block is an array of CUDA Threads that execute concurrently together and can cooperate and communicate via Shared Memory and barrier synchronization. A Thread Block has a Thread Block ID within its Grid.
	Sequence of SIMD Lane operations	CUDA Thread	A vertical cut of a thread of SIMD instructions corresponding to one element executed by one SIMD Lane. Result is stored depending on mask. AMD and OpenCL call a CUDA Thread a “work item.”	A CUDA Thread is a lightweight thread that executes a sequential program and can cooperate with other CUDA Threads executing in the same Thread Block. A CUDA Thread has a thread ID within its Thread Block.

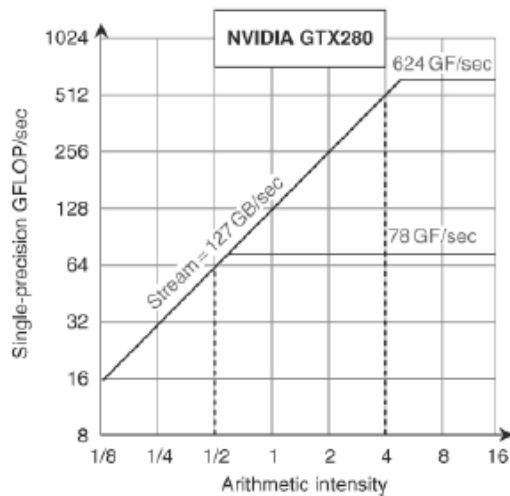
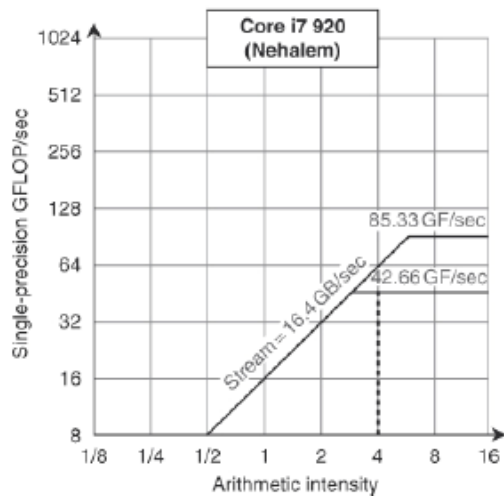
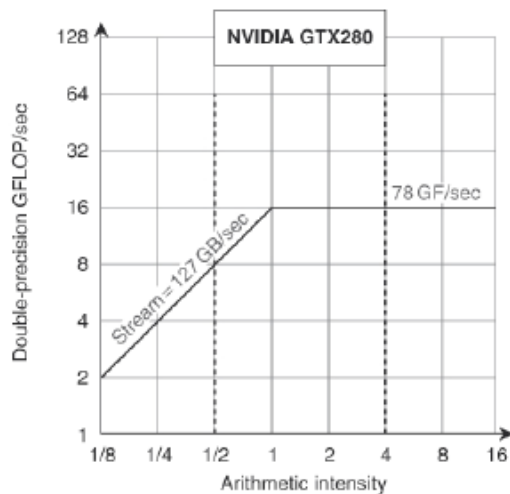
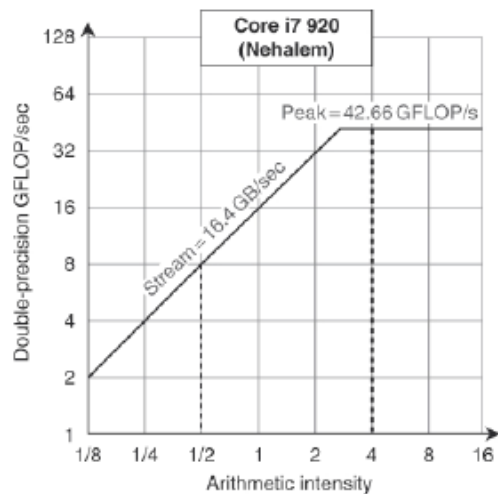
Machine object	A Thread of SIMD instructions	Warp	A traditional thread, but it contains just SIMD instructions that are executed on a multithreaded SIMD Processor. Results are stored depending on a per-element mask. AMD name is “wavefront.”	A warp is a set of parallel CUDA Threads (e.g., 32) that execute the same instruction together in a multithreaded SIMT/SIMD Processor.
	SIMD instruction	PTX instruction	A single SIMD instruction executed across the SIMD Lanes. AMD name is “AMDIL” or “FSAIL” instruction.	A PTX instruction specifies an instruction executed by a CUDA Thread.

Figure 4.24 Conversion from terms used in this chapter to official NVIDIA/CUDA and AMD jargon. OpenCL names are given in the book definition.

Type	More descriptive name used in this book	Official CUDA/NVIDIA term	Book definition and AMD and OpenCL terms	Official CUDA/NVIDIA definition
Processing hardware	Multithreaded SIMD processor	Streaming multi-processor	Multithreaded SIMD Processor that executes thread of SIMD instructions, independent of other SIMD Processors. Both AMD and OpenCL call it a “compute unit.” However, the CUDA Programmer writes program for one lane rather than for a “vector” of multiple SIMD Lanes.	A streaming multiprocessor (SM) is a multithreaded SIMT/ SIMD Processor that executes warps of CUDA Threads. A SIMT program specifies the execution of one CUDA Thread, rather than a vector of multiple SIMD Lanes.
	Thread block scheduler	Giga thread engine	Assigns multiple bodies of vectorized loop to multithreaded SIMD Processors. AMD name is “Ultra-Threaded Dispatch Engine”.	Distributes and schedules thread blocks of a grid to streaming multiprocessors as resources become available.
	SIMD Thread scheduler	Warp scheduler	Hardware unit that schedules and issues threads of SIMD instructions when they are ready to execute; includes a scoreboard to track SIMD Thread execution. AMD name is “Work Group Scheduler”.	A warp scheduler in a streaming multiprocessor schedules warps for execution when their next instruction is ready to execute.
	SIMD Lane	Thread processor	Hardware SIMD Lane that executes the operations in a thread of SIMD instructions on a single element. Results are stored depending on mask. OpenCL calls it a “processing element.” AMD name is also “SIMD Lane”.	A thread processor is a datapath and register file portion of a streaming multiprocessor that executes operations for one or more lanes of a warp.

Memory hardware	GPU Memory	Global Memory	DRAM memory accessible by all multithreaded SIMD Processors in a GPU. OpenCL calls it “Global Memory.”	Global memory is accessible by all CUDA Threads in any thread block in any grid; implemented as a region of DRAM, and may be cached.
	Private Memory	Local Memory	Portion of DRAM memory private to each SIMD Lane. Both AMD and OpenCL call it “Private Memory.”	Private “thread-local” memory for a CUDA Thread; implemented as a cached region of DRAM.
	Local Memory	Shared Memory	Fast local SRAM for one multithreaded SIMD Processor, unavailable to other SIMD Processors. OpenCL calls it “Local Memory.” AMD calls it “Group Memory”.	Fast SRAM memory shared by the CUDA Threads composing a thread block, and private to that thread block. Used for communication among CUDA Threads in a thread block at barrier synchronization points.
	SIMD Lane registers	Registers	Registers in a single SIMD Lane allocated across body of vectorized loop. AMD also calls them “Registers”.	Private registers for a CUDA Thread; implemented as multithreaded register file for certain lanes of several warps for each thread processor.

Figure 4.25 Conversion from terms used in this chapter to official NVIDIA/CUDA and AMD jargon. Note that our descriptive terms “Local Memory” and “Private Memory” use the OpenCL terminology. NVIDIA uses SIMT, single-instruction multiple-thread, rather than SIMD, to describe a streaming multiprocessor. SIMT is preferred over SIMD because the per-thread branching and control flow are unlike any SIMD machine.



Loop-Level Parallelism

- Focuses on determining whether data accesses in later iterations are dependent on data values produced in earlier iterations
 - Loop-carried dependence
- Example 1:
for (i=999; i>=0; i=i-1)
 $x[i] = x[i] + s;$
- No loop-carried dependence

Loop-Level Parallelism

- Example 2:

```
for (i=0; i<100; i=i+1) {  
    A[i+1] = A[i] + C[i]; /* S1 */  
    B[i+1] = B[i] + A[i+1]; /* S2 */  
}
```

- S1 and S2 use values computed by S1 in previous iteration
- S2 uses value computed by S1 in same iteration

Loop-Level Parallelism

- Example 3:

```
for (i=0; i<100; i=i+1) {  
    A[i] = A[i] + B[i]; /* S1 */  
    B[i+1] = C[i] + D[i]; /* S2 */  
}
```

- S1 uses value computed by S2 in previous iteration but dependence is not circular so loop is parallel

- Transform to:

```
A[0] = A[0] + B[0];  
for (i=0; i<99; i=i+1) {  
    B[i+1] = C[i] + D[i];  
    A[i+1] = A[i+1] + B[i+1];  
}  
B[100] = C[99] + D[99];
```


Loop-Level Parallelism

- Example 4:

```
for (i=0;i<100;i=i+1) {  
    A[i] = B[i] + C[i];  
    D[i] = A[i] * E[i];  
}
```

- No loop-carried dependence

- Example 5:

```
for (i=1;i<100;i=i+1) {  
    Y[i] = Y[i-1] + Y[i];  
}
```

- Loop-carried dependence in the form of *recurrence*

Finding dependencies

- Assume that a 1- D array index i is *affine*:
 - $a \times i + b$ (with constants a and b)
- An index in an n - D array index is *affine* if it is affine in each dimension
- Assume:
 - Store to $a \times i + b$, then
 - Load from $c \times i + d$
 - i runs from m to n
 - Dependence exists if:
 - Given j, k such that $m \leq j \leq n, m \leq k \leq n$
 - Store to $a \times j + b$, load from $c \times k + d$, and $a \times j + b = c \times k + d$

Finding dependencies

- Generally cannot determine at compile time
- Test for absence of a dependence:
 - GCD test:
 - If a dependency exists, $\text{GCD}(c,a)$ must evenly divide $(d-b)$
- Example:

```
for (i=0; i<100; i=i+1) {  
    X[2*i+3] = X[2*i] * 5.0;  
}
```
- Answer: $a=2, b=3, c=2, d=0 \rightarrow \text{GCD}(c,a)=2, d-b=-3 \rightarrow$ no dependence possible.

Finding dependencies

■ Example 2:

```
for (i=0; i<100; i=i+1) {  
    Y[i] = X[i] / c; /* S1 */  
    X[i] = X[i] + c; /* S2 */  
    Z[i] = Y[i] + c; /* S3 */  
    Y[i] = c - Y[i]; /* S4 */  
}
```

```
for (i=0; i<100; i=i+1) {  
    t[i] = X[i] / c;  
    X1[i] = X[i] + c;  
    Z[i] = T[i] + c;  
    Y[i] = c - T[i];  
}
```

■ Watch for antidependencies and output dependencies:

- RAW: $S1 \rightarrow S3$, $S1 \rightarrow S4$ on $Y[i]$, not loop-carried
- WAR: $S1 \rightarrow S2$ on $X[i]$; $S3 \rightarrow S4$ on $Y[i]$
- WAW: $S1 \rightarrow S4$ on $Y[i]$

Reductions

- Reduction Operation:

```
for (i=9999; i>=0; i=i-1)  
    sum = sum + x[i] * y[i];
```

- Transform to...

```
for (i=9999; i>=0; i=i-1)  
    sum [i] = x[i] * y[i];  
for (i=9999; i>=0; i=i-1)  
    finalsum = finalsum + sum[i];
```

- Do on p processors:

```
for (i=999; i>=0; i=i-1)  
    finalsum[p] = finalsum[p] + sum[i+1000*p];
```

- Note: assumes associativity!